CAREER TRANSITION STUDY GUIDE

From Control-M Admin to Data Engineer

An extensive study guide with code snippets and embedded flashcards, for a professional with 5 years in Control-M administration and software development

SQL · Python · Data Modelling · Warehouses · Pipelines · Airflow · dbt · Spark · Kafka · Cloud · Data Quality · DevOps · Portfolio Projects · Interview Prep

Includes a Control-M → Airflow concept bridge, a 16-week roadmap and spaced-repetition flashcards throughout.



Contents

How to Use This Guide . . . 5
What is inside . . . 5
Spaced repetition with the flashcards . . . 5
Recommended local setup . . . 5
Your Unfair Advantage: Control-M to Data Engineering . . . 7
Concept mapping: what you already know . . . 7
The real gaps to close . . . 8
Self-assessment (fill in now, then again at week 16) . . . 8
The 16-Week Roadmap . . . 10
Data Engineering Foundations . . . 12
The data lifecycle . . . 12
Batch vs streaming . . . 12
ETL vs ELT . . . 12
OLTP vs OLAP . . . 12
Lake, warehouse, lakehouse and the medallion architecture . . . 13
Roles around the data engineer . . . 13
SQL for Data Engineers . . . 15
The practice schema . . . 15
Logical order of execution . . . 15
Joins . . . 16
Aggregation, GROUP BY, HAVING . . . 16
CTEs and subqueries . . . 17
Window functions: the most important topic . . . 17
Writing data: INSERT, UPDATE, MERGE (upsert) . . . 19
DDL vs DML, and DELETE vs TRUNCATE vs DROP . . . 19
Performance and query tuning . . . 20
SQL interview practice . . . 20
Python for Data Engineering . . . 23
Core language features you must be fluent in . . . 23
Streaming a large file with generators . . . 23
Calling REST APIs: pagination, retries, rate limits . . . 24
pandas essentials (and when not to use it) . . . 24
Configuration, logging and structure . . . 25
Testing with pytest . . . 26
Python interview questions . . . 26
Data Modelling . . . 28
Normalisation (OLTP side) . . . 28

Dimensional modelling (Kimball) . . . 28
Fact table types . . . 29
Dimensions . . . 29
Slowly Changing Dimensions (SCD) . . . 29
Star vs snowflake vs One Big Table vs Data Vault . . . 30
Storage, File Formats and Warehouses . . . 33
Row vs columnar storage . . . 33
File formats . . . 33
Partitioning, clustering and the small-files problem . . . 34
Cloud data warehouses compared . . . 34
Open table formats (lakehouse) . . . 34
Building Reliable Pipelines: Core Patterns . . . 37
Idempotency: the number one principle . . . 37
Full vs incremental loads . . . 37
Change Data Capture (CDC) . . . 38
Backfills, late data and reprocessing . . . 38
Schema evolution, bad records and retries . . . 38
Orchestration with Apache Airflow . . . 40
Architecture . . . 40
Side by side: a Control-M flow and the same flow in Airflow . . . 40
TaskFlow API (modern Pythonic style) . . . 41
Key concepts checklist . . . 42
Best practices interviewers look for . . . 42
Transformations with dbt . . . 45
How dbt works . . . 45
Distributed Processing with Apache Spark . . . 48
Architecture . . . 48
Lazy evaluation: transformations vs actions . . . 48
A complete PySpark job . . . 48
Performance tuning: what interviewers ask . . . 49
Streaming and Apache Kafka . . . 52
Kafka core concepts . . . 52
Delivery semantics . . . 52
Stream processing concepts . . . 53
Cloud Platforms for Data Engineering . . . 55
Service map . . . 55
A reference batch architecture on AWS . . . 55
Working with object storage in Python . . . 55
Security and cost basics . . . 56

Data Quality, Observability and Governance . . . 58
Six dimensions of data quality . . . 58
Where to test . . . 58
Observability: the five pillars . . . 59
Governance, security and privacy . . . 59
DevOps for Data: Git, Docker, CI/CD, IaC . . . 61
Git workflow for data teams . . . 61
Docker: package a pipeline . . . 61
CI/CD with GitHub Actions . . . 61
Infrastructure as Code with Terraform . . . 62
Linux and Shell for Data Engineers . . . 64
Portfolio Projects . . . 65
Project 1: Batch ELT pipeline, end to end . . . 65
Project 2: Control-M to Airflow migration tool (your signature project) . . . 65
Project 3: Streaming pipeline . . . 66
Read a real repository . . . 66
Resume, Interviews and Landing the Role . . . 68
Reframing your resume . . . 68
Typical interview loop . . . 68
Your 'why data engineering' pitch . . . 69
STAR stories to prepare . . . 69
Pipeline system design framework . . . 69
Questions to ask the interviewer . . . 70
Glossary . . . 71
Appendix: Rapid-Fire Review Deck . . . 72
Final checklist before applying . . . 73

CHAPTER 1

How to Use This Guide

You already run production. You already know what a 2 a.m. failed job feels like. This guide turns five years of Control-M administration and software development into a data engineering profile, one concept at a time.

This guide is written for a professional with about **five years of Control-M administration plus some software development**, and no formal data engineering title yet. It doesn't assume any big-data background, but it does assume you know scheduling, job dependencies, Linux and production support, because you do.

What is inside

Element	What it looks like	How to use it
Concept sections	Plain-language explanations	Read once slowly, then skim before interviews.
Code blocks	Syntax-highlighted SQL, Python, PySpark, YAML, Bash	Type them out. Do not copy-paste; the typing is the learning.
Control-M Bridge	Amber boxes	Link each new idea to something you already did in Control-M. These become interview stories.
Flashcards	Blue (front) / white (back) cards	Cover the right column with a sheet of paper, answer aloud, reveal. Mark the ones you miss.
Interview Tip / Pitfall	Green / red boxes	What interviewers actually probe, and the traps that lose offers.
Practice	Purple boxes	Small exercises. Do them, and push the answers to GitHub.

Spaced repetition with the flashcards

Use the Leitner system. It's the cheapest memory technique that actually works:

1. Every card starts in **Box 1**. Review Box 1 every day.
2. Answer right: the card moves up a box. Box 2 gets reviewed every 3 days, Box 3 weekly, and Box 4 every two weeks.
3. Answer wrong: the card goes back to Box 1, however high it had reached.
4. Write the card number (e.g. **#045**) in a notebook or spreadsheet with its current box. Twenty minutes a day is enough.

> **KEY IDEA**
> Studying for a career switch is a marathon. Aim for **about 10 hours a week for 16 weeks**: about 1 hour on weekdays and about 2.5 hours on each weekend day. Consistency beats intensity.

Recommended local setup

one-time setup · BASH

```bash
# 1. Python 3.11+ with an isolated environment
python3 -m venv ~/de-env && source ~/de-env/bin/activate
pip install pandas pyarrow duckdb requests pytest pyspark==3.5.* dbt-duckdb
# 2. Docker Desktop (for Postgres, Kafka and Airflow in containers)
docker run -d --name pg -e POSTGRES_PASSWORD=pg -p 5432:5432 postgres:16
# 3. Free practice databases / clouds
#   - DuckDB: a warehouse in a file, perfect for SQL practice (no server needed)
#   - BigQuery sandbox: free, no credit card, 10 GB storage / 1 TB queries per month
#   - Snowflake / Databricks: 30-day / community trials
```

CHAPTER 2

Your Unfair Advantage: Control-M to Data Engineering

Most career switchers into data engineering start from analytics and have never run production. You start from production and need to learn the data side. That is the easier direction.

A data engineer builds and runs the systems that move data from where it is produced to where it is used, reliably, on time and correct. Roughly half of that is **workload automation and operations**, which you have done for five years. The other half is **data**: SQL, modelling, distributed processing and cloud warehouses. This guide concentrates on that half.

Concept mapping: what you already know

Control-M concept	Data engineering equivalent	Where to learn it
Job	Task (Airflow), op/asset (Dagster), node (dbt)	Ch. 9
Folder / SMART Folder / Sub-folder	DAG (directed acyclic graph) / TaskGroup	Ch. 9
In / Out conditions	Task dependencies (>>), Datasets / data-aware scheduling	Ch. 9
File Watcher job	Sensor (FileSensor, S3KeySensor, GCSObjectExistenceSensor)	Ch. 9
Calendars, rule-based calendars	Cron schedules, Timetables	Ch. 9
Order date (ODATE), %%ODATE	Logical date / data interval, {{ ds }}	Ch. 9
Rerun / Force OK / Hold / Free	Clear task / Mark success / Pause DAG	Ch. 9
Quantitative resources	Pools, max_active_tasks, concurrency slots	Ch. 9
Control resources (exclusive)	max_active_runs=1, mutex patterns	Ch. 9
Agents / agentless hosts	Executors and workers (Celery, Kubernetes)	Ch. 9
SLA Management, BIM	SLAs, freshness checks, data observability	Ch. 14
Automation API (jobs-as-code JSON)	DAGs as Python in Git, CI/CD	Ch. 9, 15
Cyclic jobs	Frequent schedules, micro-batches, streaming	Ch. 12
Variables %%VAR, auto-edit	Jinja templating, Airflow Variables, params	Ch. 9
Control-M for Databases / Hadoop / ETL plug-ins	Operators and provider packages	Ch. 9
Archived job output / sysout	Task logs, centralised logging, lineage	Ch. 14

CONTROL-M BRIDGE · Turn the table into talking points

Every row in this table is a sentence for your interviews: "In Control-M I managed X; the Airflow equivalent is Y and here is how they differ." Hiring managers are looking for people who understand **why** orchestration exists. You can explain failure handling, reruns, dependencies and SLAs from first-hand experience, which most junior candidates can't.

The real gaps to close

Skill	Why it matters	Priority
SQL (advanced)	About 60% of day-to-day DE work. Every interview has a SQL round.	Critical
Python for data	Ingestion, APIs, transformations, testing, glue code.	Critical
Data modelling	Star schemas, SCDs, grain. Separates engineers from scripters.	High
Cloud warehouse	BigQuery / Snowflake / Redshift / Databricks: where data lives.	High
Orchestration (Airflow)	You already have about 70% of this. Learn the Python API.	Medium (head start)
dbt	The industry-standard SQL transformation tool.	High
Spark	Distributed processing for big volumes. Asked in most mid-size+ roles.	High
Streaming (Kafka)	Real-time pipelines. Often a plus, sometimes required.	Medium
Data quality / governance	Tests, contracts, lineage, PII.	Medium
Docker, Git, CI/CD, Terraform	You probably know parts. Polish them.	Medium

Self-assessment (fill in now, then again at week 16)

Area	Week 0 (1-5)	Week 8 (1-5)	Week 16 (1-5)
SQL: joins, aggregation			
SQL: window functions, CTEs			
Python: files, APIs, pandas			
Dimensional modelling			
Airflow DAG authoring			
dbt			
Spark / PySpark			
Kafka / streaming			
One cloud platform			
Git, Docker, CI/CD			

✦ FLASHCARDS · Career framing cover the right column, answer aloud, then reveal

CARD 001 · FRONT **What does a data engineer do, in one sentence?**	**BACK** Builds and operates reliable, scalable systems that collect, store, transform and serve data so analysts, data scientists and applications can trust and use it.
CARD 002 · FRONT **What is the Airflow equivalent of a Control-M Folder?**	**BACK** A **DAG**. Jobs map to **tasks**; in/out conditions map to task dependencies (`a >> b`).
CARD 003 · FRONT **What is the Airflow equivalent of a Control-M File Watcher?**	**BACK** A **Sensor**, e.g. FileSensor, S3KeySensor or GCSObjectExistenceSensor. Use `mode='reschedule'` or deferrable sensors so they don't hold a worker slot.

CARD 004 · FRONT	BACK
What is the Airflow equivalent of Control-M's ODATE?	The **logical date** / data interval: the period a run is responsible for. In templates it's {{ ds }} or {{ data_interval_start }}.
CARD 005 · FRONT	**BACK**
Which Control-M feature maps to Airflow pools?	**Quantitative resources**. Both cap how many jobs can use a shared resource (e.g. DB connections) at once.
CARD 006 · FRONT	**BACK**
Your biggest transferable strength?	Production operations: dependency management, failure handling, reruns, SLAs, monitoring, change management, incident response, and talking to stakeholders when things break.

CHAPTER 3

The 16-Week Roadmap

A realistic plan at about 10 hours a week. Every phase ends with something you can show.

Week	Focus	Deliverable (push to GitHub)
1-2	SQL fundamentals: SELECT, joins, GROUP BY, NULLs, CTEs	50 solved problems (LeetCode SQL 50 / DataLemur / StrataScratch)
3	Advanced SQL: window functions, dedupe, MERGE, gaps-and-islands	README of 20 'pattern' queries with explanations
4	Python for data: files, generators, requests, pandas, logging, pytest	API-to-CSV ingestion script with tests
5	Data modelling: Kimball, star schema, grain, SCD types	Star schema diagram + DDL for a retail dataset
6	Storage and warehouses: Parquet, partitioning, BigQuery/Snowflake	Load public data to BigQuery sandbox; partitioned + clustered table
7	Pipeline patterns: incremental loads, idempotency, CDC, backfills	Incremental loader with watermark table
8	Airflow (your head start): DAGs, sensors, XCom, pools, backfills	Dockerised Airflow running Project 1 end-to-end
9	dbt: models, tests, snapshots, incremental, docs	dbt project with staging/marts layers + tests + docs site
10-11	Spark / PySpark: DataFrames, joins, partitions, skew, Spark SQL	PySpark job on 10M+ rows; write-up of tuning choices
12	Streaming: Kafka concepts, producers/consumers, Structured Streaming	Kafka to Spark streaming demo in docker-compose
13	Cloud deep dive (pick ONE: AWS, GCP or Azure) + IAM + cost	Project 1 deployed on cloud; Terraform for buckets/datasets
14	Data quality, observability, CI/CD, Docker	GitHub Actions running pytest + dbt build on every PR
15	Portfolio polish: Control-M to Airflow converter (Project 2)	Blog post / LinkedIn article about your migration tool
16	Interview prep: SQL drills, system design, STAR stories, resume	Resume v2, 6 STAR stories, 3 mock interviews

INTERVIEW TIP

Apply from **week 10**, not week 16. Interviews are a skill in themselves, and the first few are practice. Internal transfer is often the fastest route: if your current company has a data team, tell them now that you're learning and offer to take on their scheduling and orchestration work.

Certification options (optional but useful for career switchers)

- **Google Cloud Professional Data Engineer**: well respected, covers BigQuery, Dataflow, Pub/Sub and Composer.
- **AWS Certified Data Engineer - Associate (DEA-C01)**: covers Glue, Redshift, Kinesis, EMR and Athena.

- **Microsoft Certified: Fabric Data Engineer Associate (DP-700)**: the successor to the retired DP-203.
- **Databricks Certified Data Engineer Associate**: Spark, Delta Lake, lakehouse.
- **SnowPro Core** (Snowflake) and the **dbt Analytics Engineering** certification.

KEY IDEA

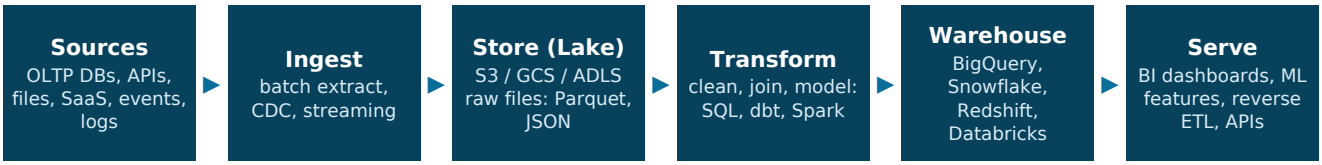
Pick **one** cloud certification that matches the job postings in your city. A certification gets you past the resume filter; projects get you through the interview.

CHAPTER 4

Data Engineering Foundations

The vocabulary and mental model everything else hangs on.

The data lifecycle



Batch vs streaming

	Batch	Streaming
Latency	Minutes to hours (e.g. nightly)	Milliseconds to seconds
Unit of work	A bounded dataset (yesterday's orders)	An unbounded stream of events
Tools	Airflow + SQL/dbt/Spark, Control-M	Kafka, Flink, Spark Structured Streaming, Kinesis, Pub/Sub
Complexity	Lower: easy to rerun and backfill	Higher: state, late data, exactly-once
Use when	Reporting, finance, most analytics	Fraud detection, real-time personalisation, monitoring

CONTROL-M BRIDGE

Every Control-M batch window you've managed (the nightly cycle, upstream-before-downstream ordering, the 6 a.m. reporting SLA) is classic **batch data engineering**. Cyclic jobs every 5 minutes are **micro-batch**, a step toward streaming.

ETL vs ELT

ETL (Extract, Transform, Load) transforms data before loading it into the warehouse, traditionally with tools like Informatica, DataStage or SSIS, because warehouse compute used to be expensive. **ELT** loads raw data first and transforms inside the warehouse with SQL (usually dbt), because cloud warehouses separate cheap storage from elastic compute. Most modern stacks are ELT: Fivetran/Airbyte to load, dbt to transform, Airflow to orchestrate.

OLTP vs OLAP

	OLTP (transactional)	OLAP (analytical)
Purpose	Run the business: place an order	Analyse the business: revenue by month
Examples	PostgreSQL, MySQL, Oracle, SQL Server	BigQuery, Snowflake, Redshift, ClickHouse
Queries	Many small reads/writes by key	Few large scans and aggregations
Storage	Row-oriented	Column-oriented
Schema	Normalised (3NF)	Denormalised (star schema)

Lake, warehouse, lakehouse and the medallion architecture

- **Data lake**: cheap object storage (S3/GCS/ADLS) holding files in any format. Flexible, but it turns into a swamp without governance.
- **Data warehouse**: structured, SQL-first, strongly governed, fast for analytics.
- **Lakehouse**: open table formats (Delta Lake, Apache Iceberg, Apache Hudi) add ACID transactions, schema enforcement and time travel to files in the lake, so you get warehouse behaviour on lake storage.

Layer	Also called	Contents	Rule
Bronze	raw / landing	Data exactly as received, plus load metadata	Append-only, never edited; lets you replay
Silver	staging / cleaned	Typed, deduplicated, conformed columns	One row per real-world entity/event
Gold	marts / presentation	Business-level facts, dimensions and aggregates	Shaped for consumers and BI

Roles around the data engineer

Role	Main focus	Main tools
Data engineer	Pipelines, platforms, reliability, scale	Python, SQL, Spark, Airflow, cloud
Analytics engineer	Clean, modelled, tested data for analysis	SQL, dbt, warehouse
Data analyst	Answer business questions, dashboards	SQL, BI (Looker, Power BI)
Data scientist / ML engineer	Models, experiments, ML in production	Python, notebooks, MLflow
Platform / DataOps engineer	Infrastructure, CI/CD, orchestration platform	Kubernetes, Terraform, Control-M/Airflow admin

INTERVIEW TIP

DataOps / Data Platform Engineer roles are a natural bridge for you. They value orchestration administration directly. Look for titles like "Data Platform Engineer", "DataOps Engineer" and "Workload Automation Engineer, Data".

✦ FLASHCARDS · Foundations cover the right column, answer aloud, then reveal

CARD 007 · FRONT **ETL vs ELT?**	**BACK** ETL transforms before loading (in an external engine). ELT loads raw data first and transforms inside the warehouse with SQL. ELT is the modern default because cloud warehouse compute is elastic and raw data is kept for replay.
CARD 008 · FRONT **OLTP vs OLAP?**	**BACK** OLTP: transactional, many small row-level reads/writes, row storage, normalised. OLAP: analytical, large scans/aggregations, columnar storage, denormalised.
CARD 009 · FRONT **Why are analytical databases columnar?**	**BACK** Queries touch few columns over many rows. Columnar storage reads only the needed columns and compresses well (similar values sit together), so there is far less I/O.

CARD 010 · FRONT	BACK
What is a lakehouse?	Lake storage (cheap files in object storage) plus an open table format (Delta, Iceberg, Hudi) that adds ACID transactions, schema enforcement and time travel.

CARD 011 · FRONT	BACK
Medallion architecture?	Bronze (raw, as received) to Silver (cleaned, typed, deduplicated) to Gold (business-level marts). Each layer increases quality and structure.

CARD 012 · FRONT	BACK
Batch vs streaming: when do you choose streaming?	When the business value depends on low latency (seconds), e.g. fraud, alerting, real-time personalisation. Otherwise batch is simpler, cheaper and easier to rerun.

CARD 013 · FRONT	BACK
What are the 'V's of big data?	Volume (size), Velocity (speed of arrival), Variety (formats), plus Veracity (trustworthiness) and Value.

CARD 014 · FRONT	BACK
Name the stages of the data lifecycle.	Generation/source, ingestion, storage, transformation, serving. Orchestration, security, quality and governance cut across all of them.

CHAPTER 5

SQL for Data Engineers

SQL is the most-used and most-tested data engineering skill. Every interview loop has a SQL round. Get fluent enough that window functions feel as natural as a Control-M dependency.

The practice schema

Every example in this chapter uses this small retail schema. Create it in DuckDB (`pip install duckdb`, then run `duckdb shop.db`) or in PostgreSQL.

practice schema · SQL

```sql
CREATE TABLE customers (
    customer_id   INTEGER PRIMARY KEY,
    name          VARCHAR,
    city          VARCHAR,
    signup_date   DATE
);
CREATE TABLE orders (
    order_id      INTEGER PRIMARY KEY,
    customer_id   INTEGER REFERENCES customers(customer_id),
    order_ts      TIMESTAMP,
    status        VARCHAR,          -- 'placed' | 'shipped' | 'cancelled'
    amount        DECIMAL(10,2)
);
INSERT INTO customers VALUES
  (1,'Asha','Pune','2024-01-05'), (2,'Ben','Leeds','2024-02-11'),
  (3,'Chen','Austin','2024-02-20'), (4,'Divya','Pune','2024-03-02');
INSERT INTO orders VALUES
  (101,1,'2024-03-01 10:00','shipped',120.00),
  (102,1,'2024-03-05 12:30','shipped', 80.00),
  (103,2,'2024-03-05 09:10','cancelled',50.00),
  (104,3,'2024-03-06 18:45','placed', 200.00),
  (105,1,'2024-03-09 08:00','shipped', 40.00),
  (106,3,'2024-03-10 14:00','shipped', 200.00);
```

Logical order of execution

SQL is written in one order and evaluated in another. Knowing the evaluation order explains most 'why doesn't this work?' errors, such as why you can't use a SELECT alias in WHERE.

Step	Clause	What happens
1	FROM / JOIN	Build the working set of rows
2	WHERE	Filter individual rows (no aggregates allowed)
3	GROUP BY	Collapse rows into groups
4	HAVING	Filter groups (aggregates allowed)
5	Window functions	Computed over the remaining rows
6	SELECT	Pick and compute output columns, assign aliases
7	DISTINCT	Remove duplicate output rows
8	ORDER BY	Sort (aliases ARE allowed here)
9	LIMIT / OFFSET	Trim the result

KEY IDEA

Some engines (BigQuery, Snowflake, DuckDB) also support **QUALIFY**, which filters on window-function results the way HAVING filters on aggregates. It is evaluated after window functions.

Joins

Join	Returns	Typical use
INNER	Only rows that match on both sides	Orders with their customer
LEFT	All left rows; NULLs where the right side has no match	All customers, with orders if any
RIGHT	Mirror of LEFT (rarely used; rewrite it as a LEFT join)	-
FULL OUTER	All rows from both sides	Reconciling two systems
CROSS	Every combination (Cartesian product)	Date spine x products
SEMI (EXISTS / IN)	Left rows that have a match, without duplicating them	Customers who ordered
ANTI (NOT EXISTS)	Left rows with no match	Customers who never ordered

anti-join and the LEFT JOIN trap · SQL

```sql
-- Customers who have never ordered (anti-join). Prefer NOT EXISTS over NOT IN:
-- NOT IN returns no rows at all if the subquery contains a single NULL.
SELECT c.*
FROM customers c
WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id);
-- LEFT JOIN trap: a filter on the right table in WHERE silently turns it into an INNER JOIN
SELECT c.name, o.order_id
FROM customers c
LEFT JOIN orders o
       ON o.customer_id = c.customer_id
      AND o.status = 'shipped'        -- keep right-table filters in the ON clause
ORDER BY c.name;
```

COMMON PITFALL

Join fan-out: joining a one-row-per-customer table to a many-rows-per-customer table multiplies rows. Summing a customer-level column afterwards double counts. Always know the **grain** (what one row means) of both sides before joining.

NULLs never equal anything, not even another NULL. Use `IS NULL`, `COALESCE` or `IS NOT DISTINCT FROM`.

Aggregation, GROUP BY, HAVING

conditional aggregation · SQL

```sql
-- Revenue per city, only for cities with more than one shipped order
SELECT c.city,
       COUNT(*)                                        AS shipped_orders,
       SUM(o.amount)                                   AS revenue,
       COUNT(DISTINCT o.customer_id)                   AS buyers,
       SUM(CASE WHEN o.amount >= 100 THEN 1 ELSE 0 END) AS big_orders  -- conditional agg
FROM orders o
JOIN customers c USING (customer_id)
WHERE o.status = 'shipped'          -- row filter: before grouping
GROUP BY c.city
HAVING COUNT(*) > 1                 -- group filter: after grouping
ORDER BY revenue DESC;
```

Note that `COUNT(*)` counts rows, `COUNT(col)` counts non-NULL values and `COUNT(DISTINCT col)` counts unique non-NULL values. Interviewers love asking about the difference.

CTEs and subqueries

A **Common Table Expression** (`WITH name AS (...)`) names an intermediate result. CTEs make SQL readable top to bottom, a bit like a chain of jobs. Recursive CTEs walk hierarchies such as org charts, bills of materials, or a job dependency graph.

recursive CTE: impact analysis · SQL

```sql
-- Recursive CTE: walk a job-dependency tree (yes, like a Control-M flow!)
WITH RECURSIVE downstream AS (
    SELECT job_name, depends_on, 1 AS depth
    FROM job_deps
    WHERE depends_on = 'LOAD_ORDERS'          -- start node
    UNION ALL
    SELECT d.job_name, d.depends_on, ds.depth + 1
    FROM job_deps d
    JOIN downstream ds ON d.depends_on = ds.job_name
)
SELECT * FROM downstream ORDER BY depth;
```

CONTROL-M BRIDGE

"Which jobs are impacted if LOAD_ORDERS fails?" is a question you've answered in Control-M's flow diagrams. In SQL it's a recursive CTE, and in data engineering it's called **lineage / impact analysis**.

Window functions: the most important topic

A window function computes a value across a set of rows related to the current row without collapsing them (unlike GROUP BY). Syntax: `FUNC() OVER (PARTITION BY ... ORDER BY ... ROWS BETWEEN ...)`.

Family	Functions	Example use
Ranking	ROW_NUMBER, RANK, DENSE_RANK, NTILE	Latest record per key; top N per group
Offset	LAG, LEAD, FIRST_VALUE, LAST_VALUE, NTH_VALUE	Change vs previous day; next event
Aggregate	SUM, AVG, COUNT, MIN, MAX ... OVER	Running totals; moving averages
Distribution	PERCENT_RANK, CUME_DIST, PERCENTILE_CONT	Percentiles

the four window patterns you must know · SQL

```sql
SELECT order_id, customer_id, order_ts, amount,
       ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_ts)      AS order_seq,
       LAG(amount)  OVER (PARTITION BY customer_id ORDER BY order_ts)      AS prev_amount,
       SUM(amount)  OVER (PARTITION BY customer_id ORDER BY order_ts
                          ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS running_total,
       AVG(amount)  OVER (ORDER BY order_ts
                          ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)         AS moving_avg_3
FROM orders
WHERE status <> 'cancelled';
```

amount	ROW_NUMBER()	RANK()	DENSE_RANK()
200	1	1	1
200	2	1	1

amount	ROW_NUMBER()	RANK()	DENSE_RANK()
120	3	3	2
80	4	4	3

Pattern: deduplicate, keeping the latest row per key

This is the single most common query in data engineering. Raw data often contains duplicates (retries, CDC updates, re-sent files).

dedupe latest record · SQL

```sql
-- Portable version
WITH ranked AS (
    SELECT *,
           ROW_NUMBER() OVER (PARTITION BY order_id
                              ORDER BY updated_at DESC, _loaded_at DESC) AS rn
    FROM raw_orders
)
SELECT * FROM ranked WHERE rn = 1;
-- BigQuery / Snowflake / DuckDB shorthand
SELECT *
FROM raw_orders
QUALIFY ROW_NUMBER() OVER (PARTITION BY order_id ORDER BY updated_at DESC) = 1;
```

COMMON PITFALL

ROW_NUMBER with a non-unique ORDER BY is **non-deterministic**: two runs can keep different rows. Always add a tie-breaker column (e.g. load timestamp or file name).

Pattern: top N per group

top-N per group · SQL

```sql
-- Top 2 orders by amount for each customer (ties included)
SELECT *
FROM (
  SELECT o.*, DENSE_RANK() OVER (PARTITION BY customer_id ORDER BY amount DESC) AS rnk
  FROM orders o
) t
WHERE rnk <= 2;
```

Pattern: gaps and islands (consecutive streaks)

To find consecutive days a user was active, subtract a row number from the date. Consecutive dates produce the same difference, which forms an 'island'.

gaps and islands · SQL

```sql
WITH d AS (
  SELECT DISTINCT user_id, CAST(event_ts AS DATE) AS day
  FROM events
),
g AS (
  SELECT user_id, day,
         day - CAST(ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY day) AS INTEGER) AS grp
  FROM d
)
SELECT user_id, MIN(day) AS streak_start, MAX(day) AS streak_end, COUNT(*) AS days
FROM g
GROUP BY user_id, grp
ORDER BY days DESC;
```

CONTROL-M BRIDGE

Same pattern, ops flavour: "find each continuous period a job was failing" from a job-run history table. Try it on exported Control-M run data as a practice exercise; it makes a great portfolio talking point.

Pattern: month-over-month growth

MoM growth with NULLIF to avoid divide-by-zero · SQL

```sql
WITH monthly AS (
  SELECT DATE_TRUNC('month', order_ts) AS month, SUM(amount) AS revenue
  FROM orders WHERE status = 'shipped'
  GROUP BY 1
)
SELECT month, revenue,
       LAG(revenue) OVER (ORDER BY month) AS prev_revenue,
       ROUND(100.0 * (revenue - LAG(revenue) OVER (ORDER BY month))
             / NULLIF(LAG(revenue) OVER (ORDER BY month), 0), 1) AS pct_growth
FROM monthly
ORDER BY month;
```

Writing data: INSERT, UPDATE, MERGE (upsert)

Pipelines constantly need to 'insert new rows, update changed rows'. MERGE does it atomically, and it's the backbone of **idempotent** loads: you can re-run it and get the same result.

upsert · SQL

```sql
MERGE INTO dw.orders AS t
USING staging.orders_batch AS s
   ON t.order_id = s.order_id
WHEN MATCHED AND s.updated_at > t.updated_at THEN
  UPDATE SET status = s.status, amount = s.amount, updated_at = s.updated_at
WHEN NOT MATCHED THEN
  INSERT (order_id, customer_id, order_ts, status, amount, updated_at)
  VALUES (s.order_id, s.customer_id, s.order_ts, s.status, s.amount, s.updated_at);
-- PostgreSQL alternative (also valid; MERGE exists in PG 15+)
INSERT INTO dw.orders AS t (order_id, status, amount, updated_at)
SELECT order_id, status, amount, updated_at FROM staging.orders_batch
ON CONFLICT (order_id) DO UPDATE
  SET status = EXCLUDED.status, amount = EXCLUDED.amount, updated_at = EXCLUDED.updated_at
  WHERE EXCLUDED.updated_at > t.updated_at;
```

COMMON PITFALL

MERGE fails (or behaves unpredictably) if the source has **duplicate keys**. Deduplicate the staging batch first with the ROW_NUMBER pattern.

DDL vs DML, and DELETE vs TRUNCATE vs DROP

Command	Type	Removes	Rollback / notes
DELETE	DML	Rows matching WHERE	Logged row by row, can roll back, fires triggers
TRUNCATE	DDL	All rows; keeps structure	Fast, minimal logging, resets identity in many DBs
DROP	DDL	The table itself (data + structure)	Gone; dependent objects break

Performance and query tuning

- **Read the plan**: `EXPLAIN` / `EXPLAIN ANALYZE` (Postgres), the Query Profile (Snowflake), Execution details (BigQuery).
- **Never SELECT * on columnar warehouses**. You pay per column scanned in BigQuery.
- **Filter on partition columns** so the engine can skip files (partition pruning). Avoid wrapping partition columns in functions: write `WHERE dt >= '2024-03-01'`, not `WHERE YEAR(dt) = 2024`.
- **Indexes** (OLTP) speed up selective lookups but slow down writes. Columnar warehouses use partitioning, clustering and sort keys instead.
- **Filter early, aggregate before joining** to shrink data before expensive joins.
- Prefer `UNION ALL` over `UNION` unless you need deduplication; UNION adds a sort/distinct step.
- **Watch out for exploding joins**: check row counts before and after each join.

reading a plan · SQL

```sql
EXPLAIN ANALYZE
SELECT c.city, SUM(o.amount)
FROM orders o JOIN customers c USING (customer_id)
WHERE o.order_ts >= DATE '2024-03-01'
GROUP BY c.city;
-- Look for: Seq Scan on a huge table (missing index / partition filter),
-- row estimates far from actual rows (stale stats -> ANALYZE),
-- Nested Loop over big inputs (consider hash join / index).
```

SQL interview practice

PRACTICE · five classic questions

Try each one before looking at the solution. Use the practice schema.

- **Q1.** Second-highest order amount overall (handle ties and the case with no second value).
- **Q2.** For each customer, the number of days between their first and second order.
- **Q3.** Customers whose shipped revenue is above the average customer revenue.
- **Q4.** Each customer's share (%) of total shipped revenue.
- **Q5.** Find duplicate rows in a table that has no primary key.

solutions · SQL

```sql
-- Q1: second-highest distinct amount (NULL if none)
SELECT MAX(amount) AS second_highest
FROM orders
WHERE amount < (SELECT MAX(amount) FROM orders);
-- or: SELECT amount FROM (SELECT DISTINCT amount, DENSE_RANK() OVER (ORDER BY amount DESC) r
--     FROM orders) WHERE r = 2;
-- Q2: days between first and second order
SELECT customer_id,
       DATE_DIFF('day', MIN(order_ts), MAX(order_ts)) AS days_to_second  -- DuckDB syntax
FROM (SELECT customer_id, order_ts,
             ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_ts) AS rn
      FROM orders) t
WHERE rn <= 2
GROUP BY customer_id
HAVING COUNT(*) = 2;
-- Q3: above-average customers
WITH rev AS (
  SELECT customer_id, SUM(amount) AS revenue
  FROM orders WHERE status = 'shipped' GROUP BY customer_id
)
SELECT * FROM rev WHERE revenue > (SELECT AVG(revenue) FROM rev);
-- Q4: share of total (window over the whole result set)
SELECT customer_id, SUM(amount) AS revenue,
       ROUND(100.0 * SUM(amount) / SUM(SUM(amount)) OVER (), 2) AS pct_of_total
FROM orders WHERE status = 'shipped'
```

solutions · SQL

```sql
GROUP BY customer_id;
-- Q5: duplicates
SELECT customer_id, order_ts, amount, COUNT(*) AS copies
FROM orders
GROUP BY customer_id, order_ts, amount
HAVING COUNT(*) > 1;
```

INTERVIEW TIP

In live SQL rounds, **talk while you type**: state the grain of each table, say which join you are choosing and why, and mention edge cases (NULLs, ties, empty groups, duplicates) before the interviewer asks.

Good free practice sites: DataLemur, StrataScratch, LeetCode (SQL 50), SQLBolt and Mode's SQL tutorial.

✦ FLASHCARDS · SQL cover the right column, answer aloud, then reveal

CARD 015 · FRONT	BACK
Logical order of SQL execution?	FROM/JOIN, WHERE, GROUP BY, HAVING, window functions, SELECT, DISTINCT, ORDER BY, LIMIT.
CARD 016 · FRONT	**BACK**
WHERE vs HAVING?	WHERE filters rows before grouping (no aggregates). HAVING filters groups after aggregation (aggregates allowed).
CARD 017 · FRONT	**BACK**
ROW_NUMBER vs RANK vs DENSE_RANK on 200, 200, 120?	ROW_NUMBER: 1,2,3. RANK: 1,1,3 (gaps after ties). DENSE_RANK: 1,1,2 (no gaps).
CARD 018 · FRONT	**BACK**
How do you keep only the latest record per key?	ROW_NUMBER() OVER (PARTITION BY key ORDER BY updated_at DESC, tiebreaker DESC), then keep rn = 1 (or use QUALIFY).
CARD 019 · FRONT	**BACK**
Why prefer NOT EXISTS over NOT IN?	If the NOT IN subquery returns any NULL, the predicate is never true and you get zero rows. NOT EXISTS handles NULLs correctly.
CARD 020 · FRONT	**BACK**
COUNT(*) vs COUNT(col) vs COUNT(DISTINCT col)?	All rows; non-NULL values of col; unique non-NULL values of col.
CARD 021 · FRONT	**BACK**
What does a LEFT JOIN with a right-table filter in WHERE do?	It removes the NULL-extended rows, effectively becoming an INNER JOIN. Put right-table filters in the ON clause instead.
CARD 022 · FRONT	**BACK**
What is a window function?	A function computed over a set of related rows (the window) defined by OVER(PARTITION BY ... ORDER BY ... frame). Unlike GROUP BY, it doesn't collapse rows.
CARD 023 · FRONT	**BACK**
ROWS vs RANGE frame?	ROWS counts physical rows (e.g. 2 PRECEDING = 2 rows back). RANGE uses value ranges, so rows with equal ORDER BY values (peers) are included together.

CARD 024 · FRONT	BACK
What is MERGE used for?	Atomic upsert: update matched rows, insert unmatched (optionally delete) in one statement. It's the basis of idempotent incremental loads.
CARD 025 · FRONT	**BACK**
What is a CTE and why use one?	A named temporary result (WITH x AS ...). Readability, reuse within a query, and recursion for hierarchies.
CARD 026 · FRONT	**BACK**
Gaps-and-islands trick?	date minus ROW_NUMBER() over (PARTITION BY entity ORDER BY date) is constant for consecutive dates; group by it to get each streak.
CARD 027 · FRONT	**BACK**
DELETE vs TRUNCATE vs DROP?	DELETE: DML, removes filtered rows, logged, can roll back. TRUNCATE: DDL, removes all rows fast, keeps structure. DROP: removes the table entirely.
CARD 028 · FRONT	**BACK**
UNION vs UNION ALL?	UNION removes duplicates (extra sort/hash step). UNION ALL keeps every row and is faster.
CARD 029 · FRONT	**BACK**
How do you avoid divide-by-zero in SQL?	Wrap the denominator in NULLIF(x, 0), so the result is NULL instead of an error. Some engines also have SAFE_DIVIDE.
CARD 030 · FRONT	**BACK**
What is partition pruning?	The engine skips partitions/files whose partition values can't match the filter. It requires filtering directly on the partition column (no functions wrapped around it).
CARD 031 · FRONT	**BACK**
What is 'grain' and why check it before a join?	Grain is what one row represents (e.g. one order line per row). Joining tables of different grain causes fan-out and double counting.
CARD 032 · FRONT	**BACK**
Correlated vs non-correlated subquery?	Correlated subqueries reference the outer row and conceptually run per row (often rewritten as joins). Non-correlated ones run once, independently.

CHAPTER 6

Python for Data Engineering

You've written software, so this chapter focuses on the Python idioms that come up in data work: streaming large files, calling APIs safely, pandas, configuration, logging and tests.

Core language features you must be fluent in

Feature	Why DEs care
dict / list / set comprehensions	Concise record transformations
Generators (yield)	Process files bigger than memory, one record at a time
Context managers (with)	Files, DB connections and locks always get closed
Decorators	Retries, timing, Airflow @task / @dag
Type hints + dataclasses	Readable, self-documenting records and configs
Exceptions (try/except/finally)	Fail loudly, retry the right errors, clean up
pathlib, datetime / zoneinfo	Paths and timezone-correct timestamps
logging (not print)	Levels, timestamps, shipping logs to central tools

Streaming a large file with generators

constant-memory file processing · PYTHON

```python
import csv
import gzip
from collections.abc import Iterator
from pathlib import Path
def read_rows(path: Path) -> Iterator[dict]:
    # Yield one dict per CSV row; works for .csv and .csv.gz without loading it all.
    opener = gzip.open if path.suffix == ".gz" else open
    with opener(path, "rt", newline="", encoding="utf-8") as f:
        yield from csv.DictReader(f)
def batched(rows: Iterator[dict], size: int = 10_000) -> Iterator[list[dict]]:
    batch = []
    for row in rows:
        batch.append(row)
        if len(batch) == size:
            yield batch
            batch = []
    if batch:
        yield batch
for chunk in batched(read_rows(Path("orders_2024.csv.gz"))):
    load_to_db(chunk)          # memory stays flat even for a 50 GB file
```

KEY IDEA

Python 3.12+ has `itertools.batched()` built in. Knowing how to write it yourself is still a common interview question.

Calling REST APIs: pagination, retries, rate limits

Most ingestion work is 'pull JSON from an API and land it somewhere'. Robust ingestion handles **pagination**, **transient failures** (HTTP 429/5xx) with exponential backoff, **timeouts**, and **secrets** that never appear in code.

paginated, retrying API client · PYTHON

```python
import logging
import os
import random
import time
import requests
log = logging.getLogger(__name__)
API = "https://api.example.com/v1/orders"
TOKEN = os.environ["ORDERS_API_TOKEN"]          # secret from env / secret manager
RETRYABLE = {429, 500, 502, 503, 504}
def get_with_retry(session, url, params, max_attempts=5):
    for attempt in range(1, max_attempts + 1):
        retry_after = None
        try:
            resp = session.get(url, params=params, timeout=30)
        except (requests.ConnectionError, requests.Timeout) as exc:
            err = repr(exc)
        else:
            if resp.status_code not in RETRYABLE:
                resp.raise_for_status()           # 4xx (not 429) -> fail fast
                return resp.json()
            err = f"HTTP {resp.status_code}"
            retry_after = resp.headers.get("Retry-After")    # server-requested wait
        if attempt == max_attempts:
            break
        wait = int(retry_after) if retry_after else 2 ** attempt + random.random()
        log.warning("attempt %s failed (%s); sleeping %.1fs", attempt, err, wait)
        time.sleep(wait)
    raise RuntimeError(f"gave up after {max_attempts} attempts: {url} ({err})")
def fetch_all(since: str):
    with requests.Session() as s:
        s.headers["Authorization"] = f"Bearer {TOKEN}"
        params = {"updated_since": since, "limit": 500}
        while True:
            page = get_with_retry(s, API, params)
            yield from page["data"]
            cursor = page.get("next_cursor")
            if not cursor:
                break
            params["cursor"] = cursor
```

INTERVIEW TIP

In interviews, say the words **idempotent**, **exponential backoff with jitter**, **cursor-based pagination** and **secrets manager**. They show production thinking, which you already have from Control-M on-call.

pandas essentials (and when not to use it)

typical pandas transform · PYTHON

```python
import pandas as pd
orders = pd.read_csv("orders.csv", parse_dates=["order_ts"],
                     dtype={"customer_id": "int64", "status": "category"})
clean = (
    orders
    .drop_duplicates(subset="order_id", keep="last")
    .query("status != 'cancelled'")
    .assign(order_date=lambda d: d["order_ts"].dt.date,
            amount=lambda d: d["amount"].fillna(0).round(2))
```

typical pandas transform · PYTHON

```
)
daily = (clean.groupby(["order_date", "customer_id"], as_index=False)
              .agg(orders=("order_id", "count"), revenue=("amount", "sum")))
customers = pd.read_parquet("customers.parquet")
report = daily.merge(customers, on="customer_id", how="left", validate="many_to_one")
report.to_parquet("daily_revenue.parquet", index=False)   # columnar, typed, compressed
```

COMMON PITFALL

pandas is **single-machine and in-memory**. Rule of thumb: fine up to a few GB. Beyond that, use **DuckDB** or **Polars** on one machine, or **Spark** / the warehouse for distributed scale.

`validate='many_to_one'` in merge raises an error on unexpected fan-out, a cheap safety net.

DuckDB: SQL on files, no server

DuckDB querying Parquet directly · PYTHON

```
import duckdb
con = duckdb.connect()   # in-memory; S3 paths need: INSTALL httpfs; LOAD httpfs;
top = con.sql('''
    SELECT customer_id, SUM(amount) AS revenue
    FROM read_parquet('s3://my-bucket/orders/dt=2024-03-*/*.parquet')
    GROUP BY 1 ORDER BY 2 DESC LIMIT 10
''').df()                # returns a pandas DataFrame
```

Configuration, logging and structure

a pipeline script the scheduler can run · PYTHON

```
import logging
import os
from dataclasses import dataclass
logging.basicConfig(
    level=os.getenv("LOG_LEVEL", "INFO"),
    format="%(asctime)s %(levelname)s %(name)s %(message)s",
)
@dataclass(frozen=True)
class Settings:
    db_url: str = os.environ.get("DB_URL", "postgresql://localhost/dev")
    batch_size: int = int(os.environ.get("BATCH_SIZE", "10000"))
    env: str = os.environ.get("ENV", "dev")
def main(run_date: str) -> int:
    settings = Settings()
    log = logging.getLogger("orders")
    log.info("run_date=%s starting load env=%s batch=%s",
             run_date, settings.env, settings.batch_size)
    ...
    return 0          # exit code 0 = success: the scheduler sees this
if __name__ == "__main__":
    import sys
    sys.exit(main(sys.argv[1]))
```

CONTROL-M BRIDGE

Exit codes, stdout/stderr and log files are how Control-M decides a job ended OK or NOTOK. The same holds when Airflow runs your script through a BashOperator or KubernetesPodOperator: **non-zero exit = failed task**. Structured logs with a run id make the 'sysout' searchable.

Testing with pytest

unit test for a transformation · PYTHON

```python
# transform.py
def normalise_status(raw: str | None) -> str:
    mapping = {"ship": "shipped", "shipped": "shipped", "cxl": "cancelled",
               "cancelled": "cancelled", "placed": "placed"}
    if raw is None:
        return "unknown"
    return mapping.get(raw.strip().lower(), "unknown")
# test_transform.py
import pytest
from transform import normalise_status
@pytest.mark.parametrize("raw, expected", [
    ("SHIP", "shipped"), (" cxl ", "cancelled"), (None, "unknown"), ("weird", "unknown"),
])
def test_normalise_status(raw, expected):
    assert normalise_status(raw) == expected
```

Python interview questions

Question	Short answer
List vs tuple vs set vs dict?	Mutable ordered / immutable ordered / unique unordered / key-value with O(1) lookup.
What is a generator?	A function using yield that produces values lazily, keeping memory constant.
Shallow vs deep copy?	A shallow copy copies the container but shares nested objects; a deep copy copies everything recursively.
What does the GIL mean for data work?	Only one thread runs Python bytecode at a time. Use threads for I/O-bound work (APIs), multiprocessing or native libraries for CPU-bound work.
*args / **kwargs?	Variable positional / keyword arguments.
How do you handle secrets?	Environment variables or a secrets manager (Vault, AWS Secrets Manager, GCP Secret Manager), never in Git.
Mutable default argument bug?	def f(x=[]) shares one list across calls. Use None and create the list inside.

✦ FLASHCARDS · Python cover the right column, answer aloud, then reveal

CARD 033 · FRONT **Why use generators in data pipelines?**	**BACK** They process one item at a time (lazy evaluation), so memory stays constant regardless of input size: ideal for big files and API pagination.
CARD 034 · FRONT **Which HTTP status codes should be retried?**	**BACK** 429 (rate limited) and 5xx (500, 502, 503, 504), plus connection errors and timeouts. Other 4xx are client errors: fail fast.
CARD 035 · FRONT **What is exponential backoff with jitter?**	**BACK** Wait 1s, 2s, 4s, 8s... between retries, plus a random offset, so many clients don't retry in lockstep and overwhelm the server.
CARD 036 · FRONT **When is pandas the wrong tool?**	**BACK** When the data doesn't fit comfortably in one machine's memory or needs distributed processing. Use DuckDB/Polars (single node) or Spark/the warehouse.

CARD 037 · FRONT	BACK
Threads vs processes in Python?	Threads suit I/O-bound work (network, disk) because the GIL is released while waiting. Processes suit CPU-bound work and give true parallelism.

CARD 038 · FRONT	BACK
Why log instead of print?	Levels, timestamps, logger names, configurable handlers and structured context, all of which central log tools can search and alert on.

CARD 039 · FRONT	BACK
What does a scheduler use to judge script success?	The process exit code: 0 = success, non-zero = failure (Control-M NOTOK, Airflow task failed).

CARD 040 · FRONT	BACK
What is a context manager?	An object used with 'with' that guarantees setup and teardown (__enter__ / __exit__), e.g. closing files or DB connections even on error.

CARD 041 · FRONT	BACK
Cursor vs offset pagination?	Offset (page=3) can skip or duplicate rows when data changes between calls. A cursor (an opaque next token / last seen id) is stable and efficient.

CARD 042 · FRONT	BACK
pytest parametrize?	@pytest.mark.parametrize runs one test function over many (input, expected) cases.

CHAPTER 7

Data Modelling

Modelling is what separates a data engineer from someone who moves files around. It decides whether every downstream query is easy and correct, or painful and wrong.

Normalisation (OLTP side)

Form	Rule	Violation example
1NF	Atomic values; no repeating groups	A 'phones' column holding '555-1, 555-2'
2NF	1NF + every non-key column depends on the whole composite key	order_lines(order_id, product_id, product_name): product_name depends only on product_id
3NF	2NF + no transitive dependencies (non-key depending on non-key)	customers(id, zip, city): city depends on zip

Normalisation removes redundancy so **writes** are safe and consistent. Analytics does the opposite: it **denormalises** so **reads** are simple and fast.

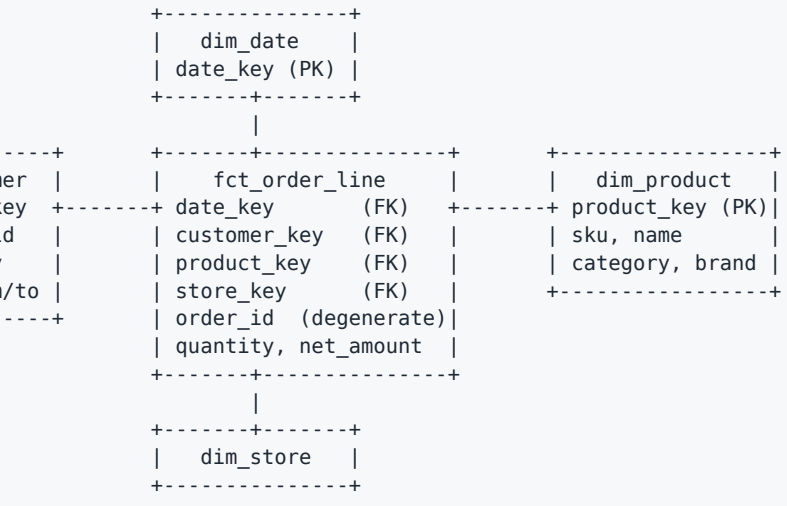
Dimensional modelling (Kimball)

Kimball's method models a business process as **facts** (measurements of events) surrounded by **dimensions** (the who, what, where and when context). The result is a **star schema**.

Kimball's four-step design process

1. **Choose the business process**: e.g. order fulfilment.
2. **Declare the grain**: exactly what one fact row represents, e.g. 'one row per order line'. This is the most important decision.
3. **Identify the dimensions**: date, customer, product, store, promotion.
4. **Identify the facts**: quantity, unit price, discount, net amount.

star schema: one fact, many dimensions · TEXT



Fact table types

Type	Grain	Example	Behaviour
Transaction	One row per event	Order line, payment, click	Insert-only; most common
Periodic snapshot	One row per entity per period	Daily account balance, month-end inventory	Regular, predictable rows; semi-additive measures
Accumulating snapshot	One row per process instance	Order lifecycle: placed, paid, shipped, delivered dates	Row updated as milestones complete
Factless	Event with no measures	Student attendance, promotion coverage	Count rows to analyse

Additivity of measures

- **Additive**: can be summed across every dimension (sales amount, quantity).
- **Semi-additive**: can be summed across some dimensions but not time (account balance, inventory level: you average them or take the last value over time).
- **Non-additive**: ratios and percentages. Store the numerator and denominator and compute the ratio at query time.

Dimensions

- **Surrogate key**: a warehouse-generated key (integer or hash) independent of the source system's **natural/business key**. It's needed for SCD Type 2 history and for combining multiple sources.
- **Conformed dimension**: shared by many fact tables with the same meaning (one dim_customer used by sales, returns and support facts), which enables drill-across.
- **Degenerate dimension**: a dimension key with no table of its own, stored on the fact (order_id, invoice number).
- **Junk dimension**: groups low-cardinality flags (is_gift, payment_type) into one small dimension.
- **Role-playing dimension**: one dimension used several ways (dim_date as order_date, ship_date, delivery_date).
- **Date dimension**: pre-built calendar with day_of_week, is_holiday, fiscal_quarter and so on. Every warehouse has one.

> **CONTROL-M BRIDGE**
> Control-M calendars (working days, holidays, fiscal periods) are essentially a **date dimension**. Your experience building rule-based calendars translates directly into building and explaining a dim_date table.

Slowly Changing Dimensions (SCD)

Dimension attributes change: a customer moves city. How do you keep history?

Type	Strategy	History?	Example
0	Retain original; never update	No	Original signup channel
1	Overwrite in place	No (current only)	Fixing a typo in a name
2	Add a new row; close the old one with valid_from / valid_to / is_current	Full	Customer city for sales-by-region over time
3	Add a 'previous_value' column	Limited (one prior)	Previous sales territory
4	Separate history (mini-dimension) table	Full, separate	Rapidly changing attributes

Type	Strategy	History?	Example
6	1 + 2 + 3 hybrid: SCD2 rows plus a current-value column on every row	Full	Report by historical or current region

SCD Type 2 · SQL

```sql
-- SCD Type 2 in two steps (works in most warehouses).
-- Step 1: close the current row for customers whose tracked attributes changed
UPDATE dim_customer d
SET    valid_to = CURRENT_TIMESTAMP, is_current = FALSE
FROM   stg_customer s
WHERE  d.customer_id = s.customer_id
  AND  d.is_current
  AND  (d.city <> s.city OR d.segment <> s.segment);
-- Step 2: insert a new current row for changed AND brand-new customers
INSERT INTO dim_customer
       (customer_key, customer_id, name, city, segment, valid_from, valid_to, is_current)
SELECT md5(s.customer_id || '|' || CAST(CURRENT_TIMESTAMP AS VARCHAR)),
       s.customer_id, s.name, s.city, s.segment,
       CURRENT_TIMESTAMP, TIMESTAMP '9999-12-31', TRUE
FROM   stg_customer s
LEFT JOIN dim_customer d
       ON d.customer_id = s.customer_id AND d.is_current
WHERE  d.customer_id IS NULL;   -- no current row: new customer, or row just closed above
```

KEY IDEA

In practice you rarely hand-write SCD2: **dbt snapshots** (Chapter 10) generate exactly this logic. But interviewers often ask you to explain or write it, so know the two steps.

querying SCD2 history · SQL

```sql
-- Point-in-time join: attach the customer's city AS IT WAS when the order happened
SELECT f.order_id, f.order_ts, d.city
FROM fct_orders f
JOIN dim_customer d
  ON d.customer_id = f.customer_id
 AND f.order_ts >= d.valid_from
 AND f.order_ts <  d.valid_to;
```

Star vs snowflake vs One Big Table vs Data Vault

Approach	Shape	Pros	Cons
Star schema	Fact + denormalised dimensions	Simple, fast joins, BI-friendly	Some redundancy in dimensions
Snowflake schema	Dimensions normalised into sub-dimensions	Less redundancy	More joins, harder for analysts
One Big Table (OBT)	Everything pre-joined into one wide table	Fastest and simplest to query in columnar DBs	Duplication; harder to maintain history
Data Vault 2.0	Hubs (keys), Links (relationships), Satellites (attributes + history)	Auditable, parallel loading, absorbs source change	Complex; still needs marts on top
Inmon (3NF EDW)	Normalised enterprise warehouse feeding data marts	Single integrated source of truth	Slow to build

INTERVIEW TIP

Asked to 'design a data model for X'? Start with: **business process, then grain, then dimensions, then facts**. State the grain out loud in one sentence. Then mention how you'd handle history (SCD2) and late-arriving data. That structure alone puts you ahead of most candidates.

✦ FLASHCARDS · Data modelling cover the right column, answer aloud, then reveal

CARD 043 · FRONT	BACK
What is the grain of a fact table?	Exactly what one row represents, e.g. 'one row per order line per day'. Declare it before choosing dimensions and facts.
CARD 044 · FRONT	**BACK**
Fact vs dimension?	Facts are numeric measurements of business events (amount, quantity). Dimensions hold descriptive context used to filter and group (customer, product, date).
CARD 045 · FRONT	**BACK**
Star vs snowflake schema?	Star: denormalised dimensions joined directly to the fact. Snowflake: dimensions normalised into sub-tables, which means more joins.
CARD 046 · FRONT	**BACK**
What is a surrogate key and why use one?	A warehouse-generated key independent of source IDs. It enables SCD2 (many rows per natural key), merges multiple sources and insulates you from source key changes.
CARD 047 · FRONT	**BACK**
SCD Type 1 vs Type 2?	Type 1 overwrites (no history). Type 2 inserts a new row and closes the old one with valid_from / valid_to / is_current (full history).
CARD 048 · FRONT	**BACK**
What is a conformed dimension?	A dimension with identical meaning and keys shared across multiple fact tables, which enables consistent cross-process analysis.
CARD 049 · FRONT	**BACK**
Three types of fact tables?	Transaction (one row per event), periodic snapshot (one row per entity per period), accumulating snapshot (one row per process instance, updated at milestones).
CARD 050 · FRONT	**BACK**
What is a semi-additive measure?	One that can be summed across some dimensions but not time, e.g. account balance or inventory level.
CARD 051 · FRONT	**BACK**
What is a degenerate dimension?	A dimension attribute stored directly on the fact with no dimension table, e.g. order_number.
CARD 052 · FRONT	**BACK**
What does 3NF guarantee?	No repeating groups (1NF), no partial dependency on a composite key (2NF), no transitive dependency between non-key columns (3NF).
CARD 053 · FRONT	**BACK**
How do you join facts to an SCD2 dimension?	On the natural key AND event_time >= valid_from AND event_time < valid_to (a point-in-time join), or store the surrogate key on the fact at load time.

CARD 054 · FRONT	BACK
Data Vault's three core table types?	Hubs (unique business keys), Links (relationships between hubs), Satellites (descriptive attributes with history).

CHAPTER 8

Storage, File Formats and Warehouses

Where data lives, and how its physical layout decides cost and speed.

Row vs columnar storage

why analytics loves columns · TEXT

```
Table:  id | name  | city   | amount
         1 | Asha  | Pune   | 120
         2 | Ben   | Leeds  |  50
Row storage (CSV, Postgres):   [1,Asha,Pune,120] [2,Ben,Leeds,50]
Columnar (Parquet, BigQuery):  [1,2] [Asha,Ben] [Pune,Leeds] [120,50]
SELECT SUM(amount) -> columnar reads ONE column block; row storage reads every byte.
```

File formats

Format	Layout	Schema	Splittable / compressible	Best for
CSV	Row, text	None (all strings)	Poor types; gzip isn't splittable	Exchange with humans and legacy systems
JSON / JSONL	Row, text	Self-describing, nested	Verbose; JSONL is splittable	APIs, semi-structured events
Avro	Row, binary	Embedded schema with evolution rules	Yes / yes	Kafka messages, write-heavy ingestion
Parquet	Columnar, binary	Embedded, typed, nested	Yes / excellent	Analytics, data lakes (the default)
ORC	Columnar, binary	Embedded, typed	Yes / excellent	Hive ecosystem

Parquet stores data in **row groups**, each holding one **column chunk** per column with min/max statistics. Engines use those statistics to skip row groups (**predicate pushdown**) and read only the requested columns (**projection pushdown**). Snappy compression is the usual default; ZSTD gives better ratios.

partitioned Parquet with PyArrow · PYTHON

```python
import pyarrow as pa
import pyarrow.dataset as ds
import pyarrow.parquet as pq
table = pa.Table.from_pylist([
    {"order_id": 101, "dt": "2024-03-01", "country": "IN", "amount": 120.0},
    {"order_id": 102, "dt": "2024-03-01", "country": "UK", "amount": 80.0},
    {"order_id": 103, "dt": "2024-03-02", "country": "IN", "amount": 50.0},
])
# Hive-style partitioning: lake/orders/dt=2024-03-01/country=IN/part-0.parquet
pq.write_to_dataset(table, root_path="lake/orders",
                    partition_cols=["dt", "country"], compression="zstd")
# Reading back with a filter: only the dt=2024-03-01 folders are opened
dataset = ds.dataset("lake/orders", format="parquet", partitioning="hive")
print(dataset.to_table(filter=ds.field("dt") == "2024-03-01").to_pandas())
```

Partitioning, clustering and the small-files problem

- **Partitioning** splits a table physically by a column, usually a date. Queries filtering on it skip whole partitions. Choose a **low-to-medium cardinality** column you filter on constantly. Never partition by user_id.
- **Clustering / sort keys / Z-ordering** sort data within partitions by frequently filtered or joined columns, so min/max statistics prune more blocks.
- **Small-files problem**: thousands of tiny files (common with streaming or over-partitioning) kill performance because every file has open and metadata overhead. Fix it with **compaction** (OPTIMIZE in Delta, rewrite_data_files in Iceberg), coalescing before writing, and aiming for roughly 128 MB to 1 GB files.

partitioned + clustered table · SQL

```sql
-- BigQuery: partition by day, cluster by common filters, force partition filters
CREATE TABLE analytics.orders
(
  order_id     INT64,
  customer_id  INT64,
  country      STRING,
  status       STRING,
  amount       NUMERIC,
  order_ts     TIMESTAMP
)
PARTITION BY DATE(order_ts)
CLUSTER BY country, customer_id
OPTIONS (require_partition_filter = TRUE, partition_expiration_days = 1095);
-- Snowflake equivalent idea: micro-partitions are automatic; add a clustering key
-- ALTER TABLE orders CLUSTER BY (TO_DATE(order_ts), country);
```

Cloud data warehouses compared

	BigQuery (GCP)	Snowflake (multi-cloud)	Redshift (AWS)	Databricks SQL
Architecture	Serverless; storage and compute separated	Virtual warehouses (compute) over shared storage	Clusters or Serverless; RA3 separates storage	Lakehouse on Delta Lake
Scaling	Automatic (slots)	Resize / multi-cluster warehouses	Resize, concurrency scaling	SQL warehouses, autoscaling
Pricing	On-demand per TB scanned or slot capacity	Credits per second of warehouse runtime	Per node-hour or RPU	DBUs + cloud compute
Layout tuning	Partition + cluster	Micro-partitions + clustering keys	Dist key + sort key	Partitioning, Z-order / liquid clustering
Standout features	BQML, streaming inserts, huge scale	Time Travel, zero-copy cloning, data sharing	Deep AWS integration, Spectrum	Unified with Spark, ML, Unity Catalog

Open table formats (lakehouse)

Plain Parquet files can't do ACID updates: two writers can corrupt a table, and a failed job leaves half-written files. Table formats add a **transaction log / metadata layer** on top of Parquet:

- **Delta Lake** (Databricks): _delta_log JSON commits; MERGE, time travel, OPTIMIZE, Z-ORDER.
- **Apache Iceberg** (Netflix origin): metadata tree of snapshots and manifests; hidden partitioning and partition evolution. Broad engine support (Spark, Trino, Snowflake, BigQuery, Athena).

- **Apache Hudi** (Uber origin): optimised for upserts and incremental pulls (copy-on-write vs merge-on-read).

Delta MERGE and time travel · PYTHON

```python
# Delta Lake with PySpark: ACID upsert + time travel
from delta.tables import DeltaTable
updates = spark.read.parquet("s3://lake/bronze/orders/dt=2024-03-10/")
target = DeltaTable.forPath(spark, "s3://lake/silver/orders")
(target.alias("t")
   .merge(updates.alias("s"), "t.order_id = s.order_id")
   .whenMatchedUpdateAll(condition="s.updated_at > t.updated_at")
   .whenNotMatchedInsertAll()
   .execute())
yesterday = (spark.read.format("delta")
                  .option("timestampAsOf", "2024-03-09")
                  .load("s3://lake/silver/orders"))     # time travel for audits / rollback
```

✦ FLASHCARDS · Storage and warehouses cover the right column, answer aloud, then reveal

CARD 055 · FRONT	BACK
Why is Parquet the default for analytics?	Columnar, compressed, typed, with embedded schema and per-row-group min/max statistics that enable column pruning and predicate pushdown. It's splittable for parallel reads.
CARD 056 · FRONT	**BACK**
Avro vs Parquet?	Avro: row-based, good for write-heavy streaming and schema evolution (Kafka). Parquet: columnar, good for read-heavy analytics.
CARD 057 · FRONT	**BACK**
Partitioning vs clustering?	Partitioning physically separates data (e.g. by date) so whole partitions are skipped. Clustering sorts data within partitions by columns so blocks can be pruned more finely.
CARD 058 · FRONT	**BACK**
What makes a good partition column?	Low-to-medium cardinality, frequently used in filters, evenly distributed. Usually a date. Avoid high-cardinality columns like user_id.
CARD 059 · FRONT	**BACK**
What is the small-files problem and its fix?	Too many tiny files add per-file open and metadata overhead and slow queries. Fix: compaction (OPTIMIZE), fewer partitions, coalesce before write, target 128 MB-1 GB files.
CARD 060 · FRONT	**BACK**
What is predicate pushdown?	Applying filters at the storage/scan layer (using file and row-group statistics or partitions) so irrelevant data is never read.
CARD 061 · FRONT	**BACK**
Name three open table formats and what they add.	Delta Lake, Apache Iceberg, Apache Hudi. They add ACID transactions, schema enforcement/evolution, time travel and efficient upserts on top of data-lake files.
CARD 062 · FRONT	**BACK**
How is BigQuery on-demand pricing calculated?	By bytes scanned per query (or reserved slot capacity). Partition filters and selecting fewer columns directly reduce cost.

CARD 063 · FRONT	BACK
Snowflake zero-copy cloning?	Creates an instant copy of a table, schema or database that shares the underlying micro-partitions. Storage is only consumed when data diverges. Great for dev/test.

CARD 064 · FRONT	BACK
Separation of storage and compute: why does it matter?	Each scales and is billed independently. Many compute clusters can read the same data, and you pay for compute only while queries run.

CARD 065 · FRONT	BACK
Redshift distribution style and sort key?	The DISTKEY decides which node holds each row (co-locate join keys to avoid redistribution). The SORTKEY orders data on disk so zone maps can skip blocks.

CHAPTER 9

Building Reliable Pipelines: Core Patterns

The design patterns that make pipelines safe to re-run, cheap to operate and easy to debug. You've seen what happens without them at 3 a.m.

Idempotency: the number one principle

A pipeline is **idempotent** if running it once or five times for the same input period produces the same result. Then reruns, retries and backfills are safe, and 'Force OK and pray' is never needed.

Non-idempotent (dangerous)	Idempotent (safe)
INSERT INTO sales SELECT ... (every rerun duplicates rows)	DELETE the target partition, then INSERT, in one transaction
Append files with random names	Overwrite the partition dt=2024-03-10/
Use NOW() / CURRENT_DATE inside logic	Use the run's logical date passed in as a parameter
Increment counters	MERGE on a business key

idempotent daily load · SQL

```sql
-- Partition overwrite pattern (run_date is a parameter, never CURRENT_DATE)
BEGIN;
DELETE FROM mart.daily_sales WHERE sales_date = :run_date;
INSERT INTO mart.daily_sales (sales_date, store_id, revenue, orders)
SELECT CAST(order_ts AS DATE), store_id, SUM(amount), COUNT(*)
FROM   silver.orders
WHERE  CAST(order_ts AS DATE) = :run_date
GROUP  BY 1, 2;
COMMIT;
```

CONTROL-M BRIDGE

Think of %%ODATE. A Control-M job ordered for 20240310 that you re-run on the 12th must still process the 10th's data. Airflow's **logical date** is the same idea, and passing it in instead of reading the clock is what makes pipelines idempotent and backfillable.

Full vs incremental loads

Strategy	How	Pros	Cons
Full refresh	Reload everything each run	Simple, always consistent	Slow and costly at scale
Incremental (watermark)	Load rows where updated_at > last high-water mark	Fast, cheap	Misses hard deletes; needs a reliable updated_at
Incremental (partition)	Reprocess only the run date's partition(s)	Idempotent, easy backfills	Late data needs a lookback window
CDC (change data capture)	Read the database's transaction log	Captures inserts, updates AND deletes, low source load	More infrastructure (Debezium, Kafka)

watermark-based incremental extract · PYTHON

```python
from datetime import datetime, timezone
import psycopg   # psycopg 3
def incremental_extract(src_dsn: str, wh_dsn: str, table: str) -> int:
    with psycopg.connect(wh_dsn) as wh:
        row = wh.execute(
            "SELECT last_value FROM etl.watermarks WHERE table_name = %s", (table,)
        ).fetchone()
        last = row[0] if row else datetime(1970, 1, 1, tzinfo=timezone.utc)
        with psycopg.connect(src_dsn) as src:
            rows = src.execute(
                f"SELECT * FROM {table} WHERE updated_at > %s ORDER BY updated_at",
                (last,),
            ).fetchall()
        if not rows:
            return 0
        load_into_staging(wh, table, rows)          # e.g. COPY into staging.<table>
        merge_staging_into_target(wh, table)        # MERGE on primary key (idempotent)
        new_mark = max(r.updated_at for r in rows)  # assumes a row_factory with attributes
        wh.execute(
            """INSERT INTO etl.watermarks (table_name, last_value) VALUES (%s, %s)
               ON CONFLICT (table_name) DO UPDATE SET last_value = EXCLUDED.last_value""",
            (table, new_mark),
        )
        # all three writes commit together when the 'with' block exits
    return len(rows)
```

COMMON PITFALL

Advance the watermark **only after** the load commits, in the same transaction if possible. Otherwise a crash between the two silently loses data.

Use **>** with a small overlap (e.g. last - 5 minutes) plus MERGE to tolerate clock skew and long-running source transactions.

Change Data Capture (CDC)



Log-based CDC reads the database's write-ahead log. Each event carries `op` (c/u/d), `before` and `after` images and a log position. Managed options include AWS DMS, GCP Datastream, Fivetran and Airbyte.

Backfills, late data and reprocessing

- **Backfill**: run the pipeline for historical intervals (e.g. after fixing a bug or adding a column). Only safe if tasks are idempotent and parameterised by date.
- **Late-arriving data**: events that show up after their period was processed. Handle them with a **lookback window** (reprocess the last N days each run) or event-time watermarks in streaming.
- **Late-arriving dimensions**: a fact arrives before its customer row exists. Insert an 'unknown' / inferred member row (key -1) and update it when the dimension arrives.

Schema evolution, bad records and retries

- **Schema drift**: sources add, remove or retype columns. Detect it (compare against the expected schema), allow additive changes automatically, and alert on breaking ones. **Data contracts** make the expected schema explicit and owned by the producer.

- **Dead-letter queue (DLQ)**: route records that fail parsing or validation to a quarantine table or topic with the error reason, instead of failing the whole batch or silently dropping them.
- **Retries**: retry transient errors (network, throttling) with backoff. Don't retry deterministic errors (bad SQL, schema mismatch); fail fast and alert.
- **Audit columns**: add _loaded_at, _source_file and _batch_id to every raw row for traceability. This is your data 'sysout'.

✦ FLASHCARDS · Pipeline patterns cover the right column, answer aloud, then reveal

CARD 066 · FRONT **What does idempotent mean for a pipeline?**	BACK Running it multiple times for the same input/interval gives the same final state: no duplicates, no drift. It makes retries and backfills safe.
CARD 067 · FRONT **Two common ways to make a load idempotent?**	BACK 1) Delete-then-insert (or overwrite) the target partition for the run date in one transaction. 2) MERGE/upsert on a business key.
CARD 068 · FRONT **Why shouldn't pipeline logic use CURRENT_DATE?**	BACK Reruns and backfills would process the wrong period. Pass the logical/run date in as a parameter instead.
CARD 069 · FRONT **Watermark incremental load: main weakness?**	BACK It can't see hard deletes and depends on a trustworthy updated_at. Late commits can be missed without an overlap window.
CARD 070 · FRONT **What is CDC and why is log-based CDC preferred?**	BACK Change Data Capture streams inserts, updates and deletes. Log-based CDC (reading the WAL/binlog, e.g. Debezium) captures deletes, has minimal source load and preserves order.
CARD 071 · FRONT **What is a backfill?**	BACK Running a pipeline for past intervals to populate or correct historical data. It requires idempotent, date-parameterised tasks.
CARD 072 · FRONT **How do you handle late-arriving facts?**	BACK Reprocess a lookback window (e.g. last 3 days) each run, or use event-time windows/watermarks in streaming; MERGE to avoid duplicates.
CARD 073 · FRONT **What is a dead-letter queue?**	BACK A place (table, topic, bucket) where invalid records are sent along with error details, so the good records keep flowing and the bad ones can be inspected and replayed.
CARD 074 · FRONT **What is a data contract?**	BACK An explicit, versioned agreement between data producer and consumer about schema, semantics, quality and SLAs, enforced in CI or at ingestion.
CARD 075 · FRONT **Late-arriving dimension: what do you do?**	BACK Load the fact with an inferred/'unknown' dimension member (e.g. key -1 or a placeholder row), then update it when the real dimension record arrives.

CHAPTER 10

Orchestration with Apache Airflow

This is your home turf. Learn the Python API and the vocabulary, and you'll be ahead of most candidates.

Architecture

Component	Role	Control-M analogue
Scheduler	Parses DAGs, decides what runs when, queues tasks	Control-M Server (scheduling engine)
Executor	How tasks run: Local, Celery, Kubernetes	Agent dispatch model
Workers	Processes or pods that execute tasks	Agents / agentless hosts
Metadata DB	Postgres/MySQL: DAG runs, task states, history	Control-M/EM database
Webserver / API server	UI: grid, graph, logs, triggers, clear/mark	Control-M Web / Monitoring domain
DAG folder / bundles	Python files defining workflows	Job definitions (Planning domain / Automation API JSON)
Triggerer	Runs deferrable (async) sensors efficiently	(no direct equivalent)

Side by side: a Control-M flow and the same flow in Airflow

Control-M Automation API (jobs-as-code), simplified · JSON

```
{
  "DWH_ORDERS_DAILY": {
    "Type": "Folder",
    "ControlmServer": "ctm-prod",
    "Application": "DWH",
    "RunAs": "etluser",
    "When": { "WeekDays": ["MON","TUE","WED","THU","FRI"], "FromTime": "0200" },
    "WAIT_ORDERS_FILE": {
      "Type": "Job:FileWatcher:Create",
      "Path": "/data/in/orders_%%ODATE.csv",
      "Host": "etl-agent-01"
    },
    "LOAD_ORDERS": {
      "Type": "Job:Command",
      "Command": "python /opt/etl/load_orders.py --date %%ODATE",
      "Host": "etl-agent-01",
      "DB_SLOT": { "Type": "Resource:Semaphore", "Quantity": "1" }
    },
    "BUILD_MART": {
      "Type": "Job:Command",
      "Command": "dbt build --select marts.sales --vars '{run_date: %%ODATE}'",
      "Host": "etl-agent-01"
    },
    "flow": { "Type": "Flow", "Sequence": ["WAIT_ORDERS_FILE", "LOAD_ORDERS", "BUILD_MART"] }
  }
}
```

the same flow as an Airflow DAG · PYTHON

```python
from datetime import datetime, timedelta
from airflow import DAG
from airflow.operators.bash import BashOperator
from airflow.sensors.filesystem import FileSensor
default_args = {
    "owner": "data-eng",
    "retries": 3,                                  # like Control-M rerun settings
    "retry_delay": timedelta(minutes=5),
    "retry_exponential_backoff": True,
    "email_on_failure": True,
    "email": ["dataops@example.com"],
}
with DAG(
    dag_id="dwh_orders_daily",                     # = Folder
    start_date=datetime(2024, 1, 1),
    schedule="0 2 * * 1-5",                        # = When: weekdays, 02:00
    catchup=False,                                 # don't auto-run missed past intervals
    max_active_runs=1,                             # = exclusive control resource
    default_args=default_args,
    tags=["dwh", "orders"],
) as dag:
    wait_orders_file = FileSensor(                 # = File Watcher job
        task_id="wait_orders_file",
        filepath="/data/in/orders_{{ ds_nodash }}.csv",   # {{ ds_nodash }} ~ %%ODATE
        poke_interval=60,
        timeout=60 * 60 * 3,
        mode="reschedule",                         # free the worker slot between pokes
    )
    load_orders = BashOperator(
        task_id="load_orders",
        bash_command="python /opt/etl/load_orders.py --date {{ ds }}",
        pool="dwh_db",                             # = quantitative resource (semaphore)
        execution_timeout=timedelta(hours=1),
    )
    build_mart = BashOperator(
        task_id="build_mart",
        bash_command="dbt build --select marts.sales --vars '{run_date: {{ ds }}}'",
    )
    wait_orders_file >> load_orders >> build_mart   # = Flow / in-out conditions
```

KEY IDEA

Airflow 3 (released 2025) is now common. Key changes: DAG authoring imports move to `airflow.sdk`; core operators such as BashOperator and FileSensor live in the `apache-airflow-providers-standard` package; `execution_date` is gone (use logical_date / data interval); SubDAGs and the old SLA feature were removed; there is DAG versioning and a new React UI. The concepts in this chapter are the same in both versions.

Check which version the job description mentions and read the matching docs.

TaskFlow API (modern Pythonic style)

TaskFlow DAG · PYTHON

```python
import pendulum
from airflow.decorators import dag, task     # Airflow 3: from airflow.sdk import dag, task
@dag(schedule="@daily", start_date=pendulum.datetime(2024, 1, 1, tz="UTC"), catchup=False)
def orders_api_pipeline():
    @task(retries=3)
    def extract(ds=None) -> str:
        rows = fetch_orders(updated_on=ds)            # your API client from Chapter 5
        path = f"s3://lake/bronze/orders/dt={ds}/orders.jsonl"
        write_jsonl(path, rows)
        return path                                    # passed via XCom (small values only!)
    @task
    def validate(path: str) -> str:
```

TaskFlow DAG · PYTHON

```
        n = count_rows(path)
        if n == 0:
            raise ValueError(f"no rows in {path}")     # fail loudly, don't load empty data
        return path
    @task
    def load(path: str) -> None:
        copy_into_warehouse(path, table="raw.orders")
    load(validate(extract()))                           # dependencies inferred from data flow
orders_api_pipeline()
```

Key concepts checklist

Concept	Meaning	Watch out for
DAG run & logical date	One execution for one data interval	A daily run for 10 March starts after the interval ends, i.e. on 11 March
catchup	Auto-create runs for missed intervals since start_date	Defaults to False in Airflow 3 (True in 2.x); set it explicitly
Backfill	Explicitly run historical intervals (CLI/UI)	Tasks must be idempotent
Operators	Templates for a unit of work (Bash, Python, SQL, KubernetesPod)	Prefer provider operators over shell scripts for cloud services
Hooks / Connections	Reusable clients + stored credentials	Never hard-code credentials; use a secrets backend
Sensors	Wait for a condition (file, partition, external task)	Use reschedule mode or deferrable sensors
XCom	Pass small metadata between tasks	Not for datasets; pass paths, not data
Pools	Limit concurrent tasks hitting a resource	= Control-M quantitative resource
Trigger rules	all_success (default), all_done, one_failed, none_failed...	Use all_done for cleanup / notification tasks
Datasets / Assets	Data-aware scheduling: run when upstream data updates	= Control-M cross-folder conditions
Dynamic task mapping	task.expand(x=list) creates N parallel tasks at runtime	Great for 'one task per file/table'

Data-aware scheduling across DAGs

Datasets (Assets in Airflow 3) · PYTHON

```
from airflow.datasets import Dataset      # Airflow 3: from airflow.sdk import Asset
orders_ready = Dataset("s3://lake/silver/orders")
# Producer DAG: a task declares that it updates the dataset
load = BashOperator(task_id="load", bash_command="...", outlets=[orders_ready])
# Consumer DAG: runs whenever the dataset is updated (like a cross-folder condition)
with DAG("sales_mart", schedule=[orders_ready], start_date=datetime(2024, 1, 1)) as dag:
    ...
```

Best practices interviewers look for

- **Keep DAG files light**: no heavy work or DB calls at the top level. The scheduler re-parses files constantly.
- **Airflow orchestrates; it doesn't process**. Push heavy compute to the warehouse, Spark or Kubernetes pods, not the Airflow worker.
- **Atomic, idempotent tasks** parameterised by the logical date.

- **Templated SQL files** and config-driven DAG generation for many similar pipelines.
- **Alerting** via on_failure_callback to Slack/PagerDuty, plus run-duration and freshness monitoring.
- **Test DAGs in CI**: an import test that fails on DAG parse errors or cycles.

DAG integrity tests · PYTHON

```python
# tests/test_dag_integrity.py: runs in CI on every pull request
from airflow.models import DagBag
def test_no_import_errors():
    bag = DagBag(dag_folder="dags/", include_examples=False)
    assert bag.import_errors == {}, f"DAG import failures: {bag.import_errors}"
def test_every_dag_has_owner_and_retries():
    bag = DagBag(dag_folder="dags/", include_examples=False)
    for dag_id, dag in bag.dags.items():
        assert dag.default_args.get("owner"), f"{dag_id} has no owner"
        assert dag.default_args.get("retries", 0) >= 1, f"{dag_id} has no retries"
```

INTERVIEW TIP

Expect: "How would you migrate jobs from Control-M to Airflow?" A strong answer:

- **Inventory**: export folders via the Automation API (JSON); classify jobs by type (command, file watcher, DB, SAP...).
- **Map** concepts (the table in Chapter 1); identify gaps such as calendars, cross-folder conditions and quantitative resources.
- **Automate** conversion with a generator script (Project 2 in Chapter 17), then hand-tune.
- **Run in parallel** (shadow mode) and compare outputs and timings, then cut over in waves by application, with rollback plans.
- **Operate**: monitoring, alerting, runbooks, SLAs and training for the support team.

CONTROL-M BRIDGE

Many enterprises keep Control-M as the enterprise scheduler and integrate it with Airflow, cloud data services and Kubernetes, and BMC offers integrations for exactly this. 'I can run both worlds' is a valuable pitch to banks, insurers and telcos.

Other orchestrators to know by name

- **Dagster**: asset-centric (you define data assets, not just tasks); strong typing and testing.
- **Prefect**: Pythonic flows, dynamic workflows, hybrid execution.
- **Cloud-native**: AWS Step Functions and MWAA (managed Airflow), GCP Cloud Composer (managed Airflow) and Workflows, Azure Data Factory / Fabric pipelines.
- **Enterprise WLA**: Control-M, Autosys, Tivoli Workload Scheduler, Stonebranch, Redwood.

✦ FLASHCARDS · Airflow cover the right column, answer aloud, then reveal

CARD 076 · FRONT **What is a DAG in Airflow?**	**BACK** A Directed Acyclic Graph: a Python-defined workflow of tasks with dependencies and no cycles, plus a schedule.
CARD 077 · FRONT **Main Airflow components?**	**BACK** Scheduler, executor, workers, metadata database, webserver/API server, DAG files, triggerer (for deferrable tasks).
CARD 078 · FRONT **What is the logical date (formerly execution_date)?**	**BACK** The start of the data interval a DAG run covers. A daily run for 10 March actually starts after that interval ends (on 11 March).

CARD 079 · FRONT	BACK
What does catchup do?	When True, the scheduler creates runs for every missed interval between start_date and now. It's often set to False to avoid surprise mass runs.
CARD 080 · FRONT	**BACK**
What is XCom and its limitation?	Cross-communication: small values passed between tasks via the metadata DB. Pass references (paths, IDs), never large datasets.
CARD 081 · FRONT	**BACK**
Sensor 'poke' vs 'reschedule' mode?	Poke holds a worker slot while sleeping between checks. Reschedule releases the slot between checks. Deferrable sensors go further and use the async triggerer.
CARD 082 · FRONT	**BACK**
What is an Airflow pool?	A named limit on concurrent task slots, used to protect shared resources (like Control-M quantitative resources).
CARD 083 · FRONT	**BACK**
Name three executors.	LocalExecutor (single machine, parallel), CeleryExecutor (distributed workers via a queue), KubernetesExecutor (one pod per task).
CARD 084 · FRONT	**BACK**
What are trigger rules?	Conditions for a task to run based on upstream states: all_success (default), all_failed, all_done, one_success, one_failed, none_failed, and others.
CARD 085 · FRONT	**BACK**
Why shouldn't heavy processing run on Airflow workers?	Airflow is an orchestrator. Heavy compute belongs in the warehouse, Spark, or isolated containers (KubernetesPodOperator) for scalability and isolation.
CARD 086 · FRONT	**BACK**
What are Datasets / Assets?	Data-aware scheduling: a DAG can be triggered when a task in another DAG updates a named dataset/asset, instead of on a time schedule.
CARD 087 · FRONT	**BACK**
What is dynamic task mapping?	Creating a variable number of task instances at runtime with .expand(), e.g. one task per file discovered.
CARD 088 · FRONT	**BACK**
Control-M Force OK equivalent in Airflow?	'Mark success' on a task instance in the UI/CLI. 'Clear' is the rerun equivalent.
CARD 089 · FRONT	**BACK**
How do you test Airflow DAGs?	DagBag import tests in CI (no import errors, no cycles), unit tests for the Python callables, and 'airflow dags test' for a local end-to-end run.

CHAPTER 11

Transformations with dbt

dbt (data build tool) lets you write transformations as SELECT statements, and it handles dependencies, materialisation, testing and documentation. It's the T in ELT at most modern companies.

How dbt works

- A **model** is a .sql file containing one SELECT. dbt wraps it in CREATE TABLE/VIEW.
- `{{ ref('model') }}` references another model; dbt builds the dependency DAG from these refs automatically.
- `{{ source('raw', 'orders') }}` references raw tables declared in YAML, which enables freshness checks and lineage.
- **Materialisations**: view, table, incremental, ephemeral (inlined CTE), plus materialized_view in some adapters.
- **Tests**: generic (unique, not_null, accepted_values, relationships), singular (custom SQL that returns failing rows), and unit tests (dbt 1.8+).
- `dbt build` runs models, tests, seeds and snapshots in DAG order.

standard project layout · TEXT

```
my_dbt_project/
  dbt_project.yml
  models/
    staging/            # 1:1 with sources: rename, cast, light cleaning  (views)
      _sources.yml
      stg_orders.sql
      stg_customers.sql
    intermediate/       # joins / business logic building blocks
      int_orders_enriched.sql
    marts/              # facts and dimensions for consumers               (tables)
      fct_orders.sql
      dim_customers.sql
      _marts.yml        # tests + docs
  snapshots/
    customers_snapshot.sql
  tests/                # singular tests
  macros/
```

sources, docs and tests (YAML) · YAML

```yaml
version: 2
sources:
  - name: raw
    schema: raw
    loaded_at_field: _loaded_at
    freshness:                              # dbt source freshness -> alert if stale
      warn_after:  {count: 6,  period: hour}
      error_after: {count: 24, period: hour}
    tables:
      - name: orders
      - name: customers
models:
  - name: fct_orders
    description: "One row per order. Grain: order_id."
    columns:
      - name: order_id
        data_tests: [unique, not_null]
      - name: status
        data_tests:
```

sources, docs and tests (YAML) · YAML

```yaml
          - accepted_values:
              values: ['placed', 'shipped', 'cancelled']
      - name: customer_id
        data_tests:
          - relationships:
              to: ref('dim_customers')
              field: customer_id
```

staging model · SQL

```sql
-- models/staging/stg_orders.sql
with source as (
    select * from {{ source('raw', 'orders') }}
),
renamed as (
    select
        cast(id as integer)              as order_id,
        cast(customer as integer)        as customer_id,
        cast(created_at as timestamp)    as order_ts,
        lower(trim(status))              as status,
        cast(amount_cents as numeric) / 100 as amount,
        _loaded_at
    from source
)
select * from renamed
```

incremental mart model (Snowflake syntax for dateadd) · SQL

```sql
-- models/marts/fct_orders.sql
{{ config(
    materialized = 'incremental',
    unique_key   = 'order_id',
    incremental_strategy = 'merge',
    on_schema_change = 'append_new_columns'
) }}
select
    o.order_id,
    o.customer_id,
    o.order_ts,
    o.status,
    o.amount,
    o._loaded_at
from {{ ref('stg_orders') }} o
{% if is_incremental() %}
  -- only new/changed rows since the last run, with a 3-day lookback for late data
  where o._loaded_at > (select dateadd('day', -3, max(_loaded_at)) from {{ this }})
{% endif %}
```

snapshot = SCD2 for free · SQL

```sql
-- snapshots/customers_snapshot.sql  -> builds an SCD Type 2 table automatically
{% snapshot customers_snapshot %}
{{ config(
    target_schema = 'snapshots',
    unique_key    = 'customer_id',
    strategy      = 'check',
    check_cols    = ['city', 'segment']
) }}
select * from {{ source('raw', 'customers') }}
{% endsnapshot %}
-- adds dbt_valid_from, dbt_valid_to, dbt_scd_id, dbt_updated_at
```

everyday commands · BASH

```
dbt deps                                  # install packages (e.g. dbt_utils)
dbt debug                                 # check connection
dbt build --select staging+               # staging models and everything downstream
dbt build --select state:modified+ --defer --state prod-artifacts/   # slim CI
dbt source freshness
dbt docs generate && dbt docs serve       # lineage graph + documentation site
```

CONTROL-M BRIDGE

dbt's `--select model+` graph selection is like ordering a Control-M job together with its downstream successors. 'Rerun from failure' is `dbt retry`.

✦ FLASHCARDS · dbt cover the right column, answer aloud, then reveal

CARD 090 · FRONT **What does ref() do in dbt?**	**BACK** Resolves to the correct schema/table for the target environment AND registers a dependency, so dbt builds models in the right order.
CARD 091 · FRONT **source() vs ref()?**	**BACK** source() points at raw tables loaded by something else (declared in YAML, supports freshness). ref() points at other dbt models.
CARD 092 · FRONT **Four dbt materialisations?**	**BACK** view, table, incremental, ephemeral (plus materialized_view on some adapters).
CARD 093 · FRONT **What does is_incremental() do?**	**BACK** Returns true when the model already exists, isn't run with --full-refresh, and is materialised incrementally, so you can filter to only new rows.
CARD 094 · FRONT **Four built-in generic tests?**	**BACK** unique, not_null, accepted_values, relationships.
CARD 095 · FRONT **What is a dbt snapshot?**	**BACK** A mechanism to capture SCD Type 2 history of a mutable source, using the 'timestamp' or 'check' strategy.
CARD 096 · FRONT **What does 'dbt build' do?**	**BACK** Runs seeds, models, snapshots and tests together in dependency order. A failing test skips downstream nodes.
CARD 097 · FRONT **Typical dbt layer structure?**	**BACK** staging (1:1 with sources, light cleaning), intermediate (reusable logic), marts (facts/dimensions for consumers).
CARD 098 · FRONT **What is slim CI in dbt?**	**BACK** In CI, build only modified models and their children (state:modified+), deferring unchanged upstream refs to production artifacts.
CARD 099 · FRONT **What is an ephemeral model?**	**BACK** A model that isn't built in the database; it's inlined as a CTE into the models that reference it.

CHAPTER 12

Distributed Processing with Apache Spark

When data outgrows one machine, Spark splits the work across a cluster. It's asked about in most mid-size and large company interviews.

Architecture

Driver SparkSession; builds the plan; DAG + task scheduler ▶ **Cluster manager** YARN, Kubernetes, Standalone, Databricks ▶ **Executors** JVMs on worker nodes; run tasks; cache data ▶ **Tasks** one per partition, run in parallel on cores

Term	Definition
Application	Your program: one driver plus its executors
Job	Triggered by each action (count, write, collect)
Stage	A set of tasks with no shuffle between them; a new stage starts at every shuffle boundary
Task	The unit of work on one partition, run on one executor core
Partition	A chunk of the data; the unit of parallelism

Lazy evaluation: transformations vs actions

Transformations (select, filter, join, groupBy) only build a **logical plan**. Nothing runs until an **action** (count, show, collect, write). Spark's **Catalyst** optimiser then rewrites the whole plan (pushes filters down, prunes columns, picks join strategies) and **Tungsten** generates efficient code.

Narrow transformations (no shuffle)	Wide transformations (shuffle)
select, filter/where, withColumn, map, union, coalesce	groupBy, join (non-broadcast), distinct, orderBy, repartition, window partitionBy
Each output partition depends on one input partition	Data is redistributed across the network by key: expensive

A complete PySpark job

read, dedupe, broadcast join, aggregate, write · PYTHON

```python
from pyspark.sql import SparkSession, Window
from pyspark.sql import functions as F
spark = (SparkSession.builder
         .appName("daily_orders")
         .config("spark.sql.adaptive.enabled", "true")          # AQE (default on in 3.2+)
         .config("spark.sql.shuffle.partitions", "200")
         .getOrCreate())
run_date = "2024-03-10"
orders = (spark.read.parquet("s3://lake/bronze/orders/")
          .where(F.col("dt") == run_date))                       # partition pruning
customers = spark.read.parquet("s3://lake/silver/customers/")
# 1) clean + dedupe: keep latest version of each order
w = Window.partitionBy("order_id").orderBy(F.col("updated_at").desc())
clean = (orders
         .withColumn("status", F.lower(F.trim("status")))
         .withColumn("rn", F.row_number().over(w))
```

read, dedupe, broadcast join, aggregate, write · PYTHON

```
         .where("rn = 1").drop("rn")
         .where(F.col("amount").isNotNull()))
# 2) enrich: customers is small -> broadcast join avoids shuffling the big side
enriched = clean.join(F.broadcast(customers.select("customer_id", "city", "segment")),
                      on="customer_id", how="left")
# 3) aggregate
daily = (enriched
         .groupBy("city", "segment")
         .agg(F.countDistinct("customer_id").alias("buyers"),
              F.sum("amount").alias("revenue"),
              F.avg("amount").alias("avg_order_value"))
         .withColumn("dt", F.lit(run_date)))
# 4) idempotent write: overwrite only this run's partition
spark.conf.set("spark.sql.sources.partitionOverwriteMode", "dynamic")
(daily.repartition(1)                                          # few rows -> one file
      .write.mode("overwrite")
      .partitionBy("dt")
      .parquet("s3://lake/gold/daily_city_revenue/"))
```

The same thing in Spark SQL

Spark SQL · PYTHON

```
orders.createOrReplaceTempView("orders")
spark.sql('''
    SELECT city, SUM(amount) AS revenue
    FROM orders o JOIN customers c USING (customer_id)
    WHERE dt = '2024-03-10'
    GROUP BY city
''').show()
```

Performance tuning: what interviewers ask

Problem	Symptom	Fix
Shuffle-heavy joins	Huge 'shuffle write' in the Spark UI; slow stages	Broadcast the small table (default threshold 10 MB; raise spark.sql.autoBroadcastJoinThreshold carefully); filter and aggregate before the join
Data skew	One task runs far longer than others in a stage	AQE skew-join handling; salting hot keys; isolate and handle hot keys separately
Wrong partition count	Too few: idle cores, OOM. Too many: scheduling overhead, tiny files	Aim for 100-200 MB per partition; tune spark.sql.shuffle.partitions; let AQE coalesce
Small output files	Thousands of KB-sized files	coalesce() / repartition() before write; compaction
Recomputation	The same lineage computed repeatedly	cache()/persist() a DataFrame reused by several actions, then unpersist()
Python UDFs	Slow row-by-row serialisation between JVM and Python	Use built-in functions; else pandas_udf (vectorised, Arrow)
Driver OOM	collect() on large data	Never collect big data; write it out instead

salting and repartition vs coalesce · PYTHON

```
# Salting a skewed key: spread the hot customer across N sub-keys
N = 16
big = big_df.withColumn("salt", (F.rand() * N).cast("int"))
small = (small_df
         .withColumn("salt", F.explode(F.array([F.lit(i) for i in range(N)]))))
joined = big.join(small, on=["customer_id", "salt"]).drop("salt")
# repartition vs coalesce
df.repartition(64, "customer_id")   # full shuffle; can increase or decrease; balances by key
```

salting and repartition vs coalesce · PYTHON

```
df.coalesce(8)                    # no full shuffle; only decreases; may be unbalanced
```

INTERVIEW TIP

Open the **Spark UI** and know your way around it: Jobs, Stages (look for task duration skew and shuffle read/write), the SQL tab (physical plan, join strategy) and Executors (GC time, memory).

Describe a tuning story as: **symptom, then evidence in the Spark UI, then the change, then the measured result**.

CONTROL-M BRIDGE

Spark's driver/executor model is like a Control-M Server dispatching to agents, except that a single job is split into thousands of parallel tasks. Long-running stragglers are the Spark version of the one job that always holds up the batch window.

✦ FLASHCARDS · Spark cover the right column, answer aloud, then reveal

CARD 100 · FRONT	BACK
Spark driver vs executor?	The driver runs your main program, builds the plan and schedules tasks. Executors are worker JVMs that run tasks on partitions and store cached data.
CARD 101 · FRONT	**BACK**
Transformation vs action?	Transformations are lazy and build the plan (filter, join). Actions trigger execution and return or write results (count, collect, write).
CARD 102 · FRONT	**BACK**
Narrow vs wide transformation?	Narrow: each output partition comes from one input partition, no shuffle (filter, select). Wide: requires a shuffle across the network (groupBy, join, distinct).
CARD 103 · FRONT	**BACK**
What creates a new stage?	A shuffle boundary (wide transformation).
CARD 104 · FRONT	**BACK**
What is a broadcast join and when do you use it?	The small table is copied to every executor so the big table isn't shuffled. Use it when one side is small (by default 10 MB, configurable).
CARD 105 · FRONT	**BACK**
What is data skew and how do you fix it?	A few keys hold most of the data, so a few tasks run very long. Fixes: AQE skew join, salting, broadcast, handling hot keys separately.
CARD 106 · FRONT	**BACK**
repartition() vs coalesce()?	repartition does a full shuffle and can increase or decrease partitions (balanced). coalesce only decreases, avoids a full shuffle, and may be unbalanced.
CARD 107 · FRONT	**BACK**
cache() vs persist()?	cache() is persist() with the default storage level (MEMORY_AND_DISK for DataFrames). persist() lets you choose the level. Both are lazy until an action.

CARD 108 · FRONT	BACK
What is AQE?	Adaptive Query Execution: re-optimises at runtime using shuffle statistics, coalescing partitions, switching join strategies and splitting skewed partitions.
CARD 109 · FRONT	**BACK**
Why avoid Python UDFs?	Each row is serialised between the JVM and Python, and Catalyst can't optimise inside them. Prefer built-in functions or vectorised pandas_udf.
CARD 110 · FRONT	**BACK**
RDD vs DataFrame?	RDD: low-level distributed objects with no schema or optimiser. DataFrame: schema'd rows optimised by Catalyst and Tungsten. Use DataFrames by default.
CARD 111 · FRONT	**BACK**
What is the Catalyst optimizer?	Spark SQL's optimiser. It turns the logical plan into an optimised physical plan (predicate pushdown, column pruning, join selection).
CARD 112 · FRONT	**BACK**
How is Spark fault tolerant?	Lineage: lost partitions are recomputed from the recorded chain of transformations. Checkpointing truncates long lineage.
CARD 113 · FRONT	**BACK**
Ideal partition size?	Roughly 100-200 MB per partition (files of 128 MB-1 GB on disk). Too small adds overhead; too big causes spills and OOM.

CHAPTER 13

Streaming and Apache Kafka

Moving from 'nightly batch' to 'events as they happen'.

Kafka core concepts

Concept	Meaning
Broker	A Kafka server. A cluster has several for scale and fault tolerance.
Topic	A named stream of records (like a table of events), e.g. orders.v1
Partition	An ordered, append-only log within a topic; the unit of parallelism. Ordering is guaranteed only within a partition.
Offset	A record's position in a partition. Consumers track the offsets they've processed.
Producer	Writes records. The record key decides the partition (same key, same partition, preserved order).
Consumer group	Consumers sharing the work. Each partition is read by exactly one consumer in the group.
Replication factor	Copies of each partition across brokers (typically 3). One leader, the rest followers (ISR = in-sync replicas).
Retention	How long records are kept (time or size), or log compaction (keep the latest value per key).
Schema Registry	Stores Avro/Protobuf/JSON schemas and enforces compatibility rules.

Delivery semantics

Semantics	How	Risk
At-most-once	Commit offset before processing	Data loss on crash
At-least-once	Process, then commit offset (the default choice)	Duplicates on retry, so make sinks idempotent
Exactly-once	Idempotent producer + transactions (read-process-write within Kafka), or at-least-once + idempotent/transactional sink	More complex; end-to-end needs sink support

Kafka producer and consumer · PYTHON

```python
# pip install confluent-kafka
import json
from confluent_kafka import Consumer, Producer
producer = Producer({"bootstrap.servers": "localhost:9092",
                     "enable.idempotence": True, "acks": "all"})
def on_delivery(err, msg):
    if err:
        print(f"delivery failed: {err}")
order = {"order_id": 107, "customer_id": 2, "amount": 75.5, "status": "placed"}
producer.produce("orders.v1", key=str(order["customer_id"]),     # key -> partition
                 value=json.dumps(order), callback=on_delivery)
producer.flush()
consumer = Consumer({"bootstrap.servers": "localhost:9092",
                     "group.id": "orders-loader",
                     "auto.offset.reset": "earliest",
                     "enable.auto.commit": False})               # commit manually
consumer.subscribe(["orders.v1"])
try:
```

Kafka producer and consumer · PYTHON

```
    while True:
        msg = consumer.poll(1.0)
        if msg is None:
            continue
        if msg.error():
            print(msg.error()); continue
        record = json.loads(msg.value())
        upsert_order(record)                    # idempotent sink (MERGE on order_id)
        consumer.commit(message=msg)            # at-least-once: commit AFTER processing
finally:
    consumer.close()
```

Stream processing concepts

- **Event time vs processing time**: when the event happened vs when the system saw it. Analytics should almost always use event time.
- **Windows**: **tumbling** (fixed, non-overlapping, e.g. every 5 minutes), **sliding/hopping** (overlapping, e.g. 10-minute window every 5 minutes), **session** (activity separated by a gap of inactivity).
- **Watermark**: how late data may arrive and still be counted. It lets the engine finalise windows and drop old state.
- **State and checkpoints**: running aggregates are stored as state and checkpointed, so a restart resumes exactly where it left off.
- **Engines**: Spark Structured Streaming (micro-batch), Apache Flink (true event-at-a-time, low latency), Kafka Streams, cloud services (Kinesis, Dataflow/Beam).

Spark Structured Streaming from Kafka · PYTHON

```
from pyspark.sql import functions as F
from pyspark.sql.types import (DoubleType, LongType, StringType, StructField,
                               StructType, TimestampType)
schema = StructType([
    StructField("order_id", LongType()), StructField("customer_id", LongType()),
    StructField("amount", DoubleType()), StructField("status", StringType()),
    StructField("event_ts", TimestampType()),
])
events = (spark.readStream.format("kafka")
          .option("kafka.bootstrap.servers", "localhost:9092")
          .option("subscribe", "orders.v1")
          .option("startingOffsets", "latest")
          .load()
          .select(F.from_json(F.col("value").cast("string"), schema).alias("o"))
          .select("o.*"))
revenue_5m = (events
              .withWatermark("event_ts", "10 minutes")             # tolerate 10 min lateness
              .groupBy(F.window("event_ts", "5 minutes"), "status")
              .agg(F.sum("amount").alias("revenue"), F.count("*").alias("orders")))
query = (revenue_5m.writeStream
         .outputMode("append")                                       # emit finalised windows
         .format("parquet")
         .option("path", "s3://lake/gold/revenue_5m/")
         .option("checkpointLocation", "s3://lake/_checkpoints/revenue_5m/")
         .trigger(processingTime="1 minute")
         .start())
```

CONTROL-M BRIDGE

A Control-M cyclic job that runs every 5 minutes and processes 'new files since last run' is conceptually a **micro-batch with a checkpoint**. Structured Streaming automates the bookkeeping (offsets, state, exactly-once file sink) that you'd otherwise build by hand.

✦ FLASHCARDS · Streaming and Kafka cover the right column, answer aloud, then reveal

CARD 114 · FRONT **Kafka topic vs partition?**	BACK A topic is a named stream. Partitions split it into ordered, append-only logs for parallelism. Ordering is only guaranteed within a partition.
CARD 115 · FRONT **How does Kafka decide the partition for a record?**	BACK By hashing the record key (same key, same partition, so per-key order is kept). Without a key, the producer spreads records across partitions.
CARD 116 · FRONT **What is a consumer group?**	BACK A set of consumers sharing a topic's partitions. Each partition goes to exactly one consumer in the group, so the maximum useful parallelism equals the partition count.
CARD 117 · FRONT **What is an offset?**	BACK The position of a record in a partition. Consumers commit offsets to remember their progress.
CARD 118 · FRONT **At-least-once vs exactly-once?**	BACK At-least-once: process, then commit, so a crash causes duplicates (make sinks idempotent). Exactly-once: idempotent producers + transactions, or an idempotent / transactional sink.
CARD 119 · FRONT **Event time vs processing time?**	BACK Event time is when the event occurred (in the payload). Processing time is when the system handled it. Use event time for correct analytics with late data.
CARD 120 · FRONT **Tumbling vs sliding vs session window?**	BACK Tumbling: fixed, non-overlapping. Sliding: fixed size, overlapping. Session: dynamic windows closed by an inactivity gap.
CARD 121 · FRONT **What is a watermark in stream processing?**	BACK A threshold of how late events may arrive. Older events are dropped, and windows before the watermark are finalised and their state freed.
CARD 122 · FRONT **Why does Kafka use replication factor 3?**	BACK Fault tolerance: with min.insync.replicas=2 and acks=all, a write survives one broker failure without data loss.
CARD 123 · FRONT **What is log compaction?**	BACK A retention mode that keeps at least the latest record per key, useful for changelog/state topics (e.g. current customer profile).
CARD 124 · FRONT **Why use a Schema Registry?**	BACK It centralises message schemas (Avro/Protobuf) and enforces compatibility, so producers can't break consumers with incompatible changes.

CHAPTER 14

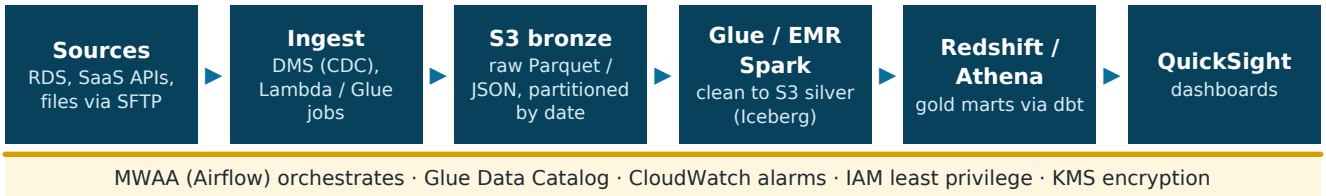
Cloud Platforms for Data Engineering

Pick one cloud and go deep. The concepts transfer; only the service names change.

Service map

Need	AWS	GCP	Azure
Object storage (lake)	S3	Cloud Storage (GCS)	ADLS Gen2 / Blob
Warehouse	Redshift	BigQuery	Synapse / Fabric Warehouse
Serverless SQL on files	Athena	BigQuery external tables / BigLake	Synapse serverless / Fabric
Managed Spark	EMR, Glue (Spark jobs)	Dataproc	Databricks, Synapse Spark, Fabric
Managed ETL / ingestion	Glue, DMS, AppFlow	Dataflow (Beam), Datastream, Data Fusion	Data Factory
Streaming	Kinesis, MSK (Kafka)	Pub/Sub, Managed Kafka	Event Hubs
Managed Airflow	MWAA	Cloud Composer	Data Factory Workflow Orchestration Manager / Fabric
Serverless functions	Lambda	Cloud Run functions	Azure Functions
Workflow (non-Airflow)	Step Functions	Workflows	Logic Apps / ADF pipelines
Catalog / governance	Glue Data Catalog, Lake Formation	Dataplex	Microsoft Purview
Secrets	Secrets Manager	Secret Manager	Key Vault
Identity	IAM roles/policies	IAM + service accounts	Entra ID + RBAC, managed identities

A reference batch architecture on AWS



Working with object storage in Python

S3 with boto3 · PYTHON

```python
import boto3
s3 = boto3.client("s3")              # credentials come from the IAM role / env, never code
s3.upload_file("daily_revenue.parquet", "acme-data-lake",
               "gold/daily_revenue/dt=2024-03-10/part-000.parquet")
paginator = s3.get_paginator("list_objects_v2")
for page in paginator.paginate(Bucket="acme-data-lake", Prefix="bronze/orders/dt=2024-03-10/"):
    for obj in page.get("Contents", []):
        print(obj["Key"], obj["Size"])
# GCP equivalent:
# from google.cloud import storage
```

S3 with boto3 · PYTHON

```python
# storage.Client().bucket("acme-data-lake").blob("gold/x.parquet").upload_from_filename("x")
```

loading and querying lake data · SQL

```sql
-- BigQuery: load Parquet from GCS, then query
LOAD DATA INTO analytics.orders_raw
FROM FILES (format = 'PARQUET', uris = ['gs://acme-data-lake/bronze/orders/dt=2024-03-10/*']);
-- Athena: query S3 files in place through a Glue catalog table
SELECT status, COUNT(*) FROM lake.orders WHERE dt = '2024-03-10' GROUP BY status;
```

Security and cost basics

- **Least privilege IAM**: each pipeline gets its own role or service account with only the buckets and datasets it needs.
- **Encryption** at rest (KMS-managed keys) and in transit (TLS). Private networking (VPC endpoints / Private Service Connect) for sensitive data.
- **No long-lived keys** in code or CI: use workload identity / OIDC federation.
- **Cost levers**: partition and cluster tables, avoid SELECT *, lifecycle policies (move old data to cold storage), auto-suspend warehouses, spot/preemptible instances for Spark, budgets and alerts.
- **Tag everything** (team, pipeline, env) so cost can be attributed.

CONTROL-M BRIDGE

You already know agents, hosts, service accounts, firewall rules and run-as users from Control-M. Cloud IAM is the same 'who can run what, where' problem, applied to APIs and data.

✦ FLASHCARDS · Cloud cover the right column, answer aloud, then reveal

CARD 125 · FRONT **AWS / GCP / Azure object storage names?**	BACK S3 / Cloud Storage (GCS) / ADLS Gen2 (Blob Storage).
CARD 126 · FRONT **AWS / GCP / Azure warehouse names?**	BACK Redshift / BigQuery / Synapse or Fabric Warehouse (plus Snowflake and Databricks on all three).
CARD 127 · FRONT **What is Athena?**	BACK AWS's serverless SQL engine (Trino-based) for querying files in S3 via the Glue Data Catalog. You pay per data scanned.
CARD 128 · FRONT **Managed Airflow services?**	BACK AWS MWAA, GCP Cloud Composer, and Azure Data Factory Workflow Orchestration Manager (Fabric also offers Apache Airflow jobs).
CARD 129 · FRONT **What is the principle of least privilege?**	BACK Grant each identity only the minimum permissions needed for its task, which limits the blast radius if it's compromised.
CARD 130 · FRONT **Name three cloud cost levers for data workloads.**	BACK Partitioning/clustering to reduce scans; auto-suspend/auto-scale compute; storage lifecycle tiers; spot instances; avoiding SELECT *.

CARD 131 · FRONT	BACK
GCP streaming ingestion and processing services?	Pub/Sub (messaging) and Dataflow (Apache Beam, unified batch + stream processing).

CARD 132 · FRONT	BACK
What is AWS Glue?	A serverless data integration service: the Glue Data Catalog (metastore), crawlers, and Spark/Python ETL jobs.

CHAPTER 15

Data Quality, Observability and Governance

A pipeline that finishes on time with wrong data is worse than one that fails. Quality is where your operations mindset shines.

Six dimensions of data quality

Dimension	Question	Example check
Completeness	Is all expected data present?	Row count within +/-20% of the 7-day average; no NULL keys
Uniqueness	Any unwanted duplicates?	order_id is unique
Validity	Do values follow the rules/format?	status IN (...); email matches a pattern; amount >= 0
Consistency	Do related datasets agree?	Sum of fact revenue = source system total
Accuracy	Does data reflect reality?	Spot-check against the source of record
Timeliness / freshness	Is it up to date?	max(_loaded_at) within the last 6 hours

Where to test

- **At ingestion**: schema and type checks, required fields, file-level checks (size > 0, expected row count from a trailer record).
- **After transformation**: dbt tests (unique, not_null, relationships), business rules, reconciliation against sources.
- **Before publishing**: block publishing a mart if critical tests fail (a 'circuit breaker'), so consumers keep yesterday's good data rather than today's bad data.
- **Continuously**: freshness, volume and distribution anomaly monitoring (data observability tools: Monte Carlo, Elementary, Soda, Great Expectations / GX).

a minimal, framework-free check runner · PYTHON

```python
from dataclasses import dataclass
import duckdb
@dataclass
class Check:
    name: str
    sql: str            # must return 0 rows / a zero count when the data is healthy
    severity: str = "error"   # 'error' blocks publish, 'warn' only alerts
CHECKS = [
    Check("orders_pk_unique",
          "SELECT order_id FROM silver.orders GROUP BY 1 HAVING COUNT(*) > 1"),
    Check("orders_amount_non_negative",
          "SELECT * FROM silver.orders WHERE amount < 0"),
    Check("orders_fresh",
          "SELECT 1 WHERE (SELECT MAX(_loaded_at) FROM silver.orders) "
          "< NOW() - INTERVAL 6 HOUR"),
    Check("orders_volume",
          '''SELECT 1 FROM (SELECT COUNT(*) n FROM silver.orders WHERE dt = $run_date) t
             WHERE n < 0.5 * (SELECT AVG(n) FROM ops.daily_counts WHERE tbl = 'orders')''',
          severity="warn"),
]
def run_checks(con: duckdb.DuckDBPyConnection, run_date: str) -> None:
    failures = []
```

a minimal, framework-free check runner · PYTHON

```
    for chk in CHECKS:
        bad = con.execute(chk.sql, {"run_date": run_date}
                          if "$run_date" in chk.sql else None).fetchall()
        status = "FAIL" if bad else "PASS"
        print(f"[{status}] {chk.name} ({len(bad)} offending rows)")
        if bad and chk.severity == "error":
            failures.append(chk.name)
    if failures:
        raise SystemExit(f"blocking data-quality failures: {failures}")   # non-zero exit
```

CONTROL-M BRIDGE

A blocking quality check is a **job that ends NOTOK so downstream jobs don't run**, exactly like a Control-M validation step that stops the flow. Freshness monitoring is Control-M SLA / BIM for data instead of jobs.

Observability: the five pillars

- **Freshness**: when was the table last updated?
- **Volume**: did row counts change unexpectedly?
- **Schema**: did columns appear, disappear or change type?
- **Distribution**: did NULL rates or value ranges shift?
- **Lineage**: what is upstream and downstream of this table, and who is affected?

Governance, security and privacy

- **Data catalog**: searchable metadata, owners, descriptions and lineage (DataHub, OpenMetadata, Unity Catalog, Dataplex, Purview, Atlan).
- **Lineage**: table- and column-level, ideally captured automatically (OpenLineage integrates with Airflow, Spark and dbt).
- **Access control**: role-based access, row-level security, column masking for PII.
- **PII handling**: classify sensitive columns; mask, hash/tokenise or drop them early (bronze to silver); encrypt; restrict raw access.
- **Regulations**: GDPR / DPDP Act (India) / CCPA: right to erasure (you must be able to delete a person's data, which shapes table design), data minimisation and retention limits.
- **Auditability**: who accessed or changed what, and when. Keep load metadata and query logs.

✦ FLASHCARDS · Data quality and governance cover the right column, answer aloud, then reveal

CARD 133 · FRONT	BACK
Six data-quality dimensions?	Completeness, uniqueness, validity, consistency, accuracy, timeliness (freshness).
CARD 134 · FRONT	**BACK**
What is a circuit breaker in a data pipeline?	A blocking check that stops downstream publication when critical tests fail, so consumers see stale-but-correct data instead of fresh-but-wrong data.
CARD 135 · FRONT	**BACK**
Five pillars of data observability?	Freshness, volume, schema, distribution, lineage.

CARD 136 · FRONT	BACK
What is data lineage and why does it matter?	The map of where data comes from and where it flows. It's used for impact analysis, root-cause debugging, compliance and trust.

CARD 137 · FRONT	BACK
How do you protect PII in a warehouse?	Classify it; minimise collection; mask, hash or tokenise; column-level security and masking policies; encryption; restricted raw zones; audit logs; retention policies.

CARD 138 · FRONT	BACK
What is OpenLineage?	An open standard for emitting lineage metadata from tools like Airflow, Spark and dbt to a backend such as Marquez or a data catalog.

CARD 139 · FRONT	BACK
Warn vs error severity for tests?	Error blocks the pipeline or publication. Warn alerts but lets data flow. Use it for soft anomalies like volume deviations.

CARD 140 · FRONT	BACK
Name three data quality / observability tools.	dbt tests, Great Expectations (GX), Soda, Monte Carlo, Elementary, Datafold.

CHAPTER 16

DevOps for Data: Git, Docker, CI/CD, IaC

Software-engineering discipline applied to data. Your development background helps here; this chapter polishes it for data teams.

Git workflow for data teams

feature-branch flow · BASH

```bash
git switch -c feat/orders-incremental        # short-lived feature branch
# ... edit models / DAGs / tests ...
git add -p                                     # review hunks before staging
git commit -m "orders: switch fct_orders to incremental merge with 3-day lookback"
git push -u origin feat/orders-incremental     # open a pull request; CI runs; peer review
# after approval: squash-merge to main -> CD deploys to prod
```

Docker: package a pipeline

Dockerfile for a pipeline job · DOCKERFILE

```dockerfile
FROM python:3.12-slim
ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY src/ ./src/
RUN useradd --create-home etl
USER etl                                       # never run as root
ENTRYPOINT ["python", "-m", "src.load_orders"]
CMD ["--date", "2024-03-10"]
```

build and run · BASH

```bash
docker build -t orders-loader:1.4.0 .
docker run --rm -e DB_URL="$DB_URL" orders-loader:1.4.0 --date 2024-03-11
# In Airflow: KubernetesPodOperator(image="registry/orders-loader:1.4.0", arguments=[...])
```

CI/CD with GitHub Actions

.github/workflows/data-ci.yml · YAML

```yaml
name: data-ci
on:
  pull_request:
    branches: [main]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - run: pip install -r requirements-dev.txt
      - name: Lint and format
        run: |
          ruff check .
          ruff format --check .
          sqlfluff lint models/ --dialect bigquery
```

.github/workflows/data-ci.yml · YAML

```yaml
      - name: Unit tests
        run: pytest -q
      - name: DAG integrity
        run: pytest tests/test_dag_integrity.py -q
      - name: dbt build on CI schema
        env:
          DBT_PROFILES_DIR: ./ci
          GCP_SA_KEY: ${{ secrets.GCP_SA_KEY }}   # better: OIDC workload identity federation
        run: dbt build --target ci --select state:modified+ --defer --state ./prod-manifest
```

Infrastructure as Code with Terraform

lake bucket + BigQuery dataset · HCL

```hcl
resource "google_storage_bucket" "lake" {
  name                        = "acme-data-lake-prod"
  location                    = "EU"
  uniform_bucket_level_access = true
  lifecycle_rule {
    condition { age = 90 }
    action {
      type          = "SetStorageClass"
      storage_class = "COLDLINE"
    }
  }
}
resource "google_bigquery_dataset" "analytics" {
  dataset_id = "analytics"
  location   = "EU"
  labels     = { team = "data-eng", env = "prod" }
}
```

CONTROL-M BRIDGE

Control-M Automation API brought **jobs-as-code**: JSON in Git, build/deploy services, environments. If you've used it (or the Control-M deploy descriptors), you've done CI/CD for workflows. Say so explicitly; it maps straight onto 'DAGs in Git + CI + Terraform'.

✦ FLASHCARDS · DevOps for data cover the right column, answer aloud, then reveal

CARD 141 · FRONT **Why containerise pipeline jobs?**	**BACK** Reproducible environments, dependency isolation, the same image in dev/CI/prod, and they're easy to run on Kubernetes (KubernetesPodOperator).
CARD 142 · FRONT **What should CI run for a data repo?**	**BACK** Lint/format (ruff, sqlfluff), unit tests, DAG import tests, dbt build/test on changed models in an isolated CI schema.
CARD 143 · FRONT **What is Infrastructure as Code?**	**BACK** Defining cloud resources (buckets, datasets, IAM, clusters) in versioned code (Terraform, Pulumi) so environments are reproducible, reviewed and auditable.
CARD 144 · FRONT **Terraform plan vs apply?**	**BACK** plan shows the diff between the code and real infrastructure without changing anything; apply executes those changes. State tracks what Terraform manages.

CARD 145 · FRONT	BACK
How should CI authenticate to the cloud?	OIDC / workload identity federation, which gives short-lived tokens, instead of long-lived JSON keys stored as secrets.

CARD 146 · FRONT	BACK
Dev / staging / prod for data: how?	Separate projects/accounts or schemas, parameterised by target (dbt targets, Airflow connections), with promotion through pull requests and CI/CD.

CHAPTER 17

Linux and Shell for Data Engineers

Probably your strongest area. A quick refresher focused on data tasks, plus a robust wrapper you can show off.

everyday one-liners · BASH

```
head -5 orders.csv; wc -l orders.csv              # peek and count
cut -d',' -f3 orders.csv | sort | uniq -c | sort -rn | head   # value frequencies
awk -F',' 'NR>1 {s+=$5} END {print s}' orders.csv  # sum a column
grep -c ',cancelled,' orders.csv                   # count matching rows
zcat big.csv.gz | tail -n +2 | split -l 1000000 - part_   # stream + split a big file
jq -c '.data[] | {id, status}' response.json       # reshape JSON
find /data/in -name 'orders_*.csv' -mtime +7 -delete  # housekeeping
du -sh /data/* | sort -h                           # what is eating disk
```

production-grade job wrapper · BASH

```
#!/usr/bin/env bash
# run_load.sh <YYYY-MM-DD> : wrapper suitable for Control-M, cron or Airflow BashOperator
set -Eeuo pipefail          # exit on error, unset vars, failures inside pipes
IFS=$'\n\t'
RUN_DATE="${1:?usage: run_load.sh YYYY-MM-DD}"
LOG_DIR="/var/log/etl"; mkdir -p "$LOG_DIR"
LOG="$LOG_DIR/load_${RUN_DATE}_$(date +%H%M%S).log"
LOCK="/tmp/run_load.lock"
exec 9>"$LOCK"
flock -n 9 || { echo "another run is active"; exit 75; }   # mutex, like a control resource
trap 'echo "FAILED at line $LINENO (exit $?)" | tee -a "$LOG"' ERR
echo "$(date -Is) start run_date=$RUN_DATE" | tee -a "$LOG"
python -m src.load_orders --date "$RUN_DATE" >>"$LOG" 2>&1
python -m src.quality_checks --date "$RUN_DATE" >>"$LOG" 2>&1
echo "$(date -Is) success" | tee -a "$LOG"
```

✦ FLASHCARDS · Linux cover the right column, answer aloud, then reveal

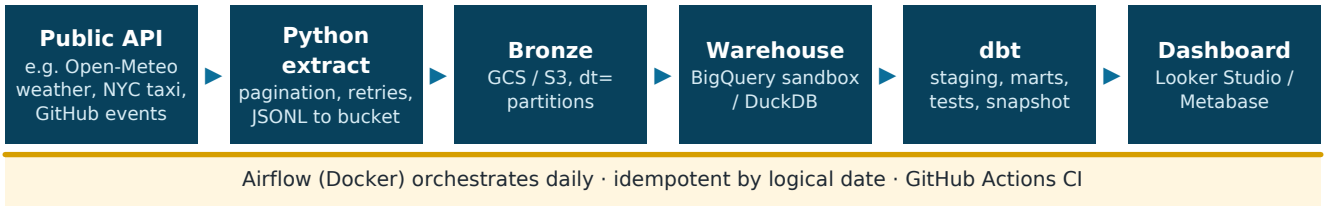
CARD 147 · FRONT	BACK
What does set -euo pipefail do?	-e exits on any error, -u errors on unset variables, -o pipefail makes a pipeline fail if any command in it fails (not just the last).
CARD 148 · FRONT	**BACK**
How do you prevent two copies of a script running at once?	Use a lock: flock on a lock file (or a scheduler mutex/pool such as a Control-M control resource or max_active_runs=1).
CARD 149 · FRONT	**BACK**
Five-field cron syntax?	minute hour day-of-month month day-of-week. E.g. '0 2 * * 1-5' = 02:00 on weekdays.
CARD 150 · FRONT	**BACK**
Quickly find the most frequent values in column 3 of a CSV?	cut -d',' -f3 file.csv | sort | uniq -c | sort -rn | head

CHAPTER 18

Portfolio Projects

Three projects that prove the skills, and one of them is something almost no other candidate can build. Put each on GitHub with a strong README.

Project 1: Batch ELT pipeline, end to end



- **Show**: idempotent reruns (run the same date twice and show identical counts), a backfill of 30 days, dbt tests catching an injected bad record, and a lineage graph screenshot from dbt docs.
- **README sections**: problem, architecture diagram, data model (grain!), how to run locally (docker compose up), design decisions and trade-offs, what you'd do at 100x scale.

Project 2: Control-M to Airflow migration tool (your signature project)

Build a converter that reads a Control-M Automation API folder definition (JSON) and generates an Airflow DAG. It shows orchestration depth, Python and migration thinking, and it's a story nobody else in the interview pool has. Start small with Command and FileWatcher jobs plus Flow sequences, then extend it.

ctm2airflow.py (starter version) · PYTHON

```
"""ctm2airflow.py - convert Control-M Automation API folders (JSON) into Airflow DAG files.
usage: python ctm2airflow.py folders.json dags/
"""
import json
import sys
from pathlib import Path
HEADER = """from datetime import datetime
from airflow import DAG
from airflow.operators.bash import BashOperator
from airflow.operators.empty import EmptyOperator
from airflow.sensors.filesystem import FileSensor
with DAG(dag_id="{dag_id}", schedule="{cron}", start_date=datetime(2024, 1, 1),
         catchup=False, max_active_runs=1, tags=["migrated-from-control-m"]) as dag:"""
DOW = {"SUN": 0, "MON": 1, "TUE": 2, "WED": 3, "THU": 4, "FRI": 5, "SAT": 6}
VARS = {"%%ODATE": "{{ ds_nodash }}", "%%$ODATE": "{{ ds_nodash }}"}
def to_cron(when: dict) -> str:
    hhmm = when.get("FromTime", "0000")
    days = ",".join(str(DOW[d]) for d in when.get("WeekDays", [])) or "*"
    return f"{int(hhmm[2:])} {int(hhmm[:2])} * * {days}"
def translate(text: str) -> str:
    for ctm, jinja in VARS.items():
        text = text.replace(ctm, jinja)
    return text
def task_line(name: str, job: dict) -> str:
    tid, kind = name.lower(), job["Type"]
    if kind == "Job:Command":
        return f"    {tid} = BashOperator(task_id={tid!r}, " \
               f"bash_command={translate(job['Command'])!r})"
    if kind.startswith("Job:FileWatcher"):
        return f"    {tid} = FileSensor(task_id={tid!r}, " \
```

ctm2airflow.py (starter version) · PYTHON

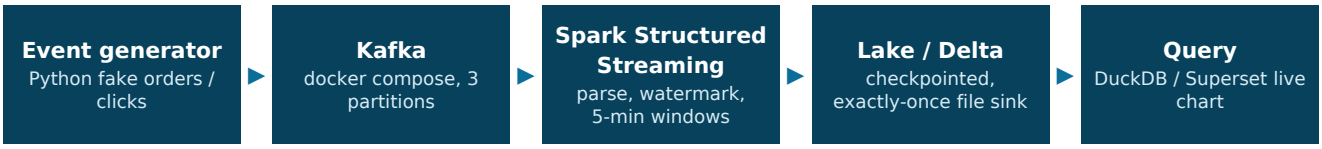
```
                f"filepath={translate(job['Path'])!r}, mode='reschedule', poke_interval=60)"
    return f"    {tid} = EmptyOperator(task_id={tid!r})  # TODO: unsupported type {kind}"
def convert(folder_name: str, folder: dict) -> str:
    jobs = {k: v for k, v in folder.items()
            if isinstance(v, dict) and v.get("Type", "").startswith("Job")}
    edges = []
    for item in folder.values():
        if isinstance(item, dict) and item.get("Type") == "Flow":
            seq = [s.lower() for s in item["Sequence"]]
            edges += zip(seq, seq[1:])
    lines = [HEADER.format(dag_id=folder_name.lower(), cron=to_cron(folder.get("When", {})))]
    lines += [task_line(n, j) for n, j in jobs.items()]
    lines += [f"    {a} >> {b}" for a, b in edges]
    return "\n".join(lines) + "\n"
if __name__ == "__main__":
    src, out = Path(sys.argv[1]), Path(sys.argv[2])
    out.mkdir(parents=True, exist_ok=True)
    for name, folder in json.loads(src.read_text()).items():
        target = out / f"{name.lower()}.py"
        target.write_text(convert(name, folder))
        print(f"wrote {target}")
```

PRACTICE · make it yours

Extensions that turn this into a standout project:

- Map **Resource:Semaphore** to Airflow **pools** and **Resource:Mutex** to max_active_runs / pools of size 1.
- Translate **in/out conditions across folders** to Datasets/Assets or ExternalTaskSensor.
- Convert **calendars** to cron expressions or custom Timetables, and flag the ones that can't be expressed.
- Produce a **migration report** (CSV/HTML): job counts by type, auto-converted vs needs-manual-work.
- Add **pytest** tests and validate the generated DAGs with a DagBag import test in CI.
- Write a LinkedIn / Medium article: "What 5 years of Control-M taught me about Airflow".

Project 3: Streaming pipeline



- **Show**: kill the streaming job mid-run, restart it, and prove there are no duplicates or gaps (checkpoints). Send deliberately late events and show the watermark behaviour.

Read a real repository

The repository this guide is stored in (**meta-ga4-pipeline**) is a compact, realistic marketing-data pipeline. After Chapters 8-10, read it as a case study:

- `marketing_ingestion_dag.py`: **config-driven DAG generation** from `sources.yaml` (one DAG per ad account), and a trailing **28-day restatement window** because the source revises historical numbers.
- `bigquery.py` / `meta_ads.py`: **MERGE-based idempotent loads**, API client and rate limiting.
- `stg_ga4_sessions.sql` and `fct_paid_performance_daily.sql`: staging-to-fact modelling, with an explicit **grain** decision (roll tracking up to ad x day before joining spend, to avoid fan-out).
- `test_meta_ads.py`: how ingestion code is unit-tested.

- `README.md`: notice how it explains why (grain, identity, restatement, attribution). Write your project READMEs the same way.

✦ FLASHCARDS · Projects cover the right column, answer aloud, then reveal

CARD 151 · FRONT **Three things every portfolio README should contain?**	**BACK** An architecture diagram, the data model with explicit grain, and design decisions / trade-offs (plus how to run it and what you'd change at scale).
CARD 152 · FRONT **How do you demonstrate idempotency in a demo?**	**BACK** Run the same logical date twice and show identical row counts and checksums; run a backfill and show no duplicates.
CARD 153 · FRONT **Why is the Control-M to Airflow converter a strong project for you?**	**BACK** It combines your unique domain expertise with Python and modern orchestration, and it maps directly to real enterprise migration work.

CHAPTER 19

Resume, Interviews and Landing the Role

Translate your experience into data engineering language, then prove it in each interview round.

Reframing your resume

Lead with **outcomes, scale and data**. Replace the bracketed values with your real numbers. Never invent metrics: interviewers will ask you about them.

Before (admin language)	After (data engineering language)
Administered Control-M for the organisation.	Operated enterprise workload orchestration (Control-M) for [2,000+] daily batch jobs feeding the [Oracle/Teradata] data warehouse and finance reporting, with a [99.x%] on-time SLA.
Monitored jobs and handled failures.	Led incident response for production data pipelines; cut mean time to recovery by [30%] through runbooks, automated reruns and dependency-aware alerting.
Created and modified job definitions.	Designed batch dependency flows and resource controls (quantitative/control resources) to shorten the nightly batch window from [6h to 4.5h].
Did Control-M upgrades.	Planned and executed Control-M [v9 to v21] upgrade and agent migration across [N] hosts with zero data-delivery misses.
Wrote scripts.	Built Python/Bash automation for file ingestion, validation and reconciliation, removing [N] hours/week of manual checks.
Worked on development tasks.	Developed [Java/Python] services and SQL for [system]; wrote unit tests and participated in code review and CI/CD.

- **Headline**: 'Data Engineer | Orchestration and Pipeline Reliability | Airflow, SQL, Python, dbt, Spark | 5 yrs production workload automation'.
- **Projects section** above older experience while you're switching, with GitHub links.
- **Keywords** from the job post (ATS filters): Airflow, ETL/ELT, SQL, Python, Spark, dbt, Kafka, BigQuery/Snowflake/Redshift, data modelling, CI/CD.

Typical interview loop

Round	What they test	How to prepare
Recruiter screen	Motivation, basics, salary, notice period	60-second 'why data engineering' pitch (below)
SQL (live)	Joins, aggregation, window functions, edge cases	Chapter 4 + 50-100 practice problems
Python / coding	Data manipulation, parsing, simple algorithms	Dicts, sorting, file/JSON parsing, LeetCode easy-medium
Data modelling	Design a schema for a business process	Process, grain, dimensions, facts, SCD
System / pipeline design	End-to-end architecture, trade-offs	The framework below
Behavioural	Ownership, incidents, collaboration	6 STAR stories from your Control-M years

Your 'why data engineering' pitch

INTERVIEW TIP · 60-second pitch template

"For five years I've run the orchestration layer that moves our company's data, with Control-M scheduling thousands of jobs that feed the warehouse and reporting. I became the person who understood why jobs failed, not just that they failed, and that pulled me into the SQL, the data and the pipeline code. Over the past months I've built [projects] with Airflow, dbt and Spark. I want to own pipelines end to end, not just schedule them, and I bring production reliability habits most new data engineers take years to learn."

STAR stories to prepare

Situation, **T**ask, **A**ction (most of the time, and say 'I', not 'we'), **R**esult (with numbers). Prepare one story for each prompt:

Prompt	Mine your Control-M experience for...
A production incident you resolved	A failed critical batch, a missed SLA, root cause, prevention
Something you automated	Scripts, Automation API, self-healing reruns, reports
A large change / migration	Version upgrade, agent migration, data centre / cloud move
Disagreement with a stakeholder	Pushing back on an unsafe change or an unrealistic schedule
A mistake you made	What broke, how you owned it, what process you changed
Learning something fast	Your data engineering transition itself!

KEY IDEA · STAR

Example (fill in your real details):

S: The finance month-end batch was regularly finishing after the 7 a.m. SLA, delaying reports to [N] executives.

T: I was asked to bring it back inside the SLA without new hardware.

A: I exported three months of run history, found that [4] long jobs ran serially without needing to, re-modelled the dependencies to run them in parallel, added a quantitative resource to protect the database, and moved a heavy reconciliation step earlier.

R: Completion moved from about 7:40 to about 5:50 a.m.; zero SLA misses for [6] months. I documented the analysis as a reusable runbook.

Pipeline system design framework

1. **Clarify requirements**: consumers, latency (batch or real-time), volume and growth, freshness SLA, accuracy needs, history and retention, compliance (PII).
2. **Sources and ingestion**: databases (CDC or batch extract), APIs, files, events. Push or pull? Full or incremental?
3. **Storage**: lake layers (bronze/silver/gold), formats (Parquet/Delta/Iceberg), partitioning.
4. **Processing**: SQL/dbt in the warehouse vs Spark vs streaming engine. Justify the choice by volume and latency.
5. **Modelling and serving**: star schema / marts, BI, APIs, ML features, reverse ETL.
6. **Orchestration**: DAG design, dependencies, retries, backfills, idempotency.
7. **Quality and observability**: tests, freshness/volume monitors, alerting, lineage, runbooks.
8. **Security and cost**: IAM, encryption, PII masking; partition pruning, auto-suspend, lifecycle rules.

9. **Scale and failure modes**: what breaks at 10x? Late data, duplicates, schema drift, source outages.

PRACTICE · system design

Worked prompt: "500 retail stores upload daily sales CSVs to SFTP by 1 a.m. Executives need a dashboard by 7 a.m. Design it."

- **Ingest:** a sensor waits for files (with a cut-off and an alert for missing stores); copy them to the bronze bucket partitioned by business date; validate row counts against trailer records.
- **Process:** load to staging; dedupe; quarantine bad rows to a DLQ table; dbt builds silver (typed, conformed) and gold (fct_sales at grain store x product x day, dim_store, dim_product SCD2, dim_date).
- **Orchestrate:** one Airflow DAG per day; per-store dynamic tasks; idempotent partition overwrite; late stores reprocessed by a 3-day lookback.
- **Quality:** blocking tests (unique keys, non-negative amounts, store coverage >= 98%) before the dashboard refresh; freshness SLA alert at 6:30 a.m.
- **Scale:** 500 stores of daily CSVs is small; a warehouse plus dbt is enough. Spark only if volume grows by orders of magnitude. Mention cost and simplicity.

Questions to ask the interviewer

- What does the current data stack look like, and what's the biggest pain point?
- How are pipelines monitored, and who is on call?
- How do you test data and deploy changes (CI/CD)?
- What would success look like for this role in the first 90 days?
- Is there an ongoing migration (e.g. from a legacy scheduler or on-prem to cloud)? (a perfect opening for your background)

✦ FLASHCARDS · Interview prep cover the right column, answer aloud, then reveal

CARD 154 · FRONT **STAR stands for?**	**BACK** Situation, Task, Action, Result. Spend most of the time on Action ('I') and quantify the Result.
CARD 155 · FRONT **First step in any system design answer?**	**BACK** Clarify requirements: consumers, latency, volume, freshness SLA, history, compliance. Then propose an architecture and discuss trade-offs.
CARD 156 · FRONT **How do you answer 'You've never had a data engineer title'?**	**BACK** Reframe: you've operated production data pipelines (orchestration, SLAs, incidents) for 5 years, and you've built [projects] covering SQL, dbt, Airflow and Spark. Give concrete examples.
CARD 157 · FRONT **Why should you mention idempotency and backfills in design answers?**	**BACK** They show you've thought about reruns and failure recovery, the operational reality most junior candidates miss.

CHAPTER 20

Glossary

Quick definitions to skim before interviews.

Term	Definition
ACID	Atomicity, Consistency, Isolation, Durability: transaction guarantees
Backfill	Running a pipeline for historical intervals
CDC	Change Data Capture: streaming row-level changes from a database log
Columnar storage	Storing each column contiguously; fast analytic scans
Data contract	Agreed, enforced schema/semantics/SLA between producer and consumer
Data mart	Subject-specific subset of the warehouse (e.g. sales mart)
DAG	Directed acyclic graph: tasks with dependencies and no cycles
Data lake	Object storage holding raw files of any format
Lakehouse	Lake + open table format giving warehouse-like ACID behaviour
DLQ	Dead-letter queue: where failed records are quarantined
ELT / ETL	Load-then-transform in the warehouse / transform-then-load
Fan-out	Row multiplication from joining across grains
Grain	What one row of a table represents
Idempotent	Same result no matter how many times it runs
Lineage	Map of data's origins and downstream dependencies
Logical date	The data interval a scheduled run is responsible for (ODATE)
Medallion	Bronze/silver/gold layering of data quality
Micro-batch	Processing a stream as frequent small batches
OLAP / OLTP	Analytical / transactional workloads
Parquet	Open columnar file format with embedded schema and statistics
Partition pruning	Skipping partitions that can't match a filter
Reverse ETL	Syncing warehouse data back into operational tools (CRM, ads)
SCD	Slowly changing dimension: strategies for attribute history
Schema evolution	Handling added/changed columns over time safely
Shuffle	Redistributing data across the network by key (Spark)
Skew	Uneven data distribution causing straggler tasks
Star schema	Fact table surrounded by denormalised dimension tables
Surrogate key	Warehouse-generated key independent of source IDs
Watermark (batch)	Last processed value used for incremental loads
Watermark (streaming)	Lateness threshold for event-time windows

APPENDIX

Appendix: Rapid-Fire Review Deck

Mixed questions across every chapter. Use this deck in the final two weeks and on the morning of each interview.

✦ FLASHCARDS · Mixed review cover the right column, answer aloud, then reveal

CARD 158 · FRONT **A daily job inserted duplicates after a rerun. Root cause and fix?**	BACK The load isn't idempotent (plain INSERT). Fix: partition overwrite (delete+insert for the run date in one transaction) or MERGE on a business key, plus a uniqueness test.
CARD 159 · FRONT **A dashboard shows revenue doubled after a new join. What happened?**	BACK Fan-out: the join was to a table at a finer grain (many rows per key). Aggregate to the same grain before joining, or fix the join key.
CARD 160 · FRONT **A Spark stage has 199 fast tasks and 1 that runs for an hour. Why?**	BACK Data skew on the partition key. Fix with AQE skew join, salting, broadcasting the other side, or handling the hot key separately.
CARD 161 · FRONT **BigQuery costs spiked. First three things to check?**	BACK SELECT * queries, missing partition filters (enable require_partition_filter), and repeated scheduled queries / dashboards scanning large tables.
CARD 162 · FRONT **The source added a column and the pipeline broke. How do you prevent it next time?**	BACK Schema-drift handling: allow additive changes (on_schema_change / mergeSchema), alert on breaking ones, and agree a data contract with the producer.
CARD 163 · FRONT **A Kafka consumer is far behind (lag growing). Options?**	BACK Add consumers up to the partition count; increase partitions; speed up processing (batching, async I/O); check for a slow sink or rebalancing loops.
CARD 164 · FRONT **Explain the difference between a data warehouse and a data lake in one breath.**	BACK A warehouse stores structured, modelled data optimised for SQL analytics; a lake stores raw files of any format cheaply. A lakehouse adds table formats to bridge the two.
CARD 165 · FRONT **What happens if an Airflow task's start_date is dynamic, like datetime.now()?**	BACK Scheduling becomes unpredictable and runs may never trigger. Always use a fixed, static start_date.
CARD 166 · FRONT **When would you pick Airflow over Control-M (and vice versa)?**	BACK Airflow: code-first data pipelines, open source, rich data integrations, Git/CI workflows. Control-M: enterprise-wide workload automation across mainframe/ERP/file transfer, mature governance and support. Many firms use both.

CARD 167 · FRONT	BACK
What does 'exactly-once' really require end to end?	Replayable source with tracked offsets, deterministic processing, and an idempotent or transactional sink committed together with the offsets/checkpoint.
CARD 168 · FRONT	**BACK**
How do you choose between dbt in the warehouse and Spark?	Data already in the warehouse and SQL-expressible: dbt. Very large volumes, unstructured data, complex non-SQL logic or ML: Spark.
CARD 169 · FRONT	**BACK**
Define SLA, SLO and SLI for a data product.	SLI: the measured metric (e.g. table freshness). SLO: the internal target (fresh by 06:30 on 99% of days). SLA: the external commitment with consequences.
CARD 170 · FRONT	**BACK**
What is the difference between a view and a materialised view?	A view stores only the query and recomputes on every read. A materialised view stores results and refreshes them (incrementally in some engines).
CARD 171 · FRONT	**BACK**
What is a surrogate key collision risk when using hashes?	Different inputs producing the same hash. With md5/sha over well-delimited natural keys it's negligible, but always include a delimiter and handle NULLs consistently.
CARD 172 · FRONT	**BACK**
Why should raw (bronze) data be immutable?	It allows replaying and rebuilding downstream layers after bugs, provides an audit trail, and decouples ingestion from transformation logic.
CARD 173 · FRONT	**BACK**
Name the four Kimball design steps.	Select the business process, declare the grain, identify dimensions, identify facts.

Final checklist before applying

Done	Item
[ ]	100+ SQL problems solved, including 30+ with window functions
[ ]	Project 1 (batch ELT) on GitHub with README, diagram and CI badge
[ ]	Project 2 (Control-M to Airflow converter) with tests and a write-up
[ ]	Project 3 (streaming) or a Spark tuning write-up
[ ]	Resume rewritten in data engineering language with real metrics
[ ]	LinkedIn headline + 'Open to work' (recruiters only) + 2 posts about your projects
[ ]	6 STAR stories written and rehearsed aloud
[ ]	System design framework practised on 3 prompts
[ ]	Every flashcard in Box 3 or higher
[ ]	One cloud certification booked or passed (optional)

KEY IDEA

You're not starting from zero. You're starting from production. Every concept in this guide builds on things you have already done at scale. Keep going.